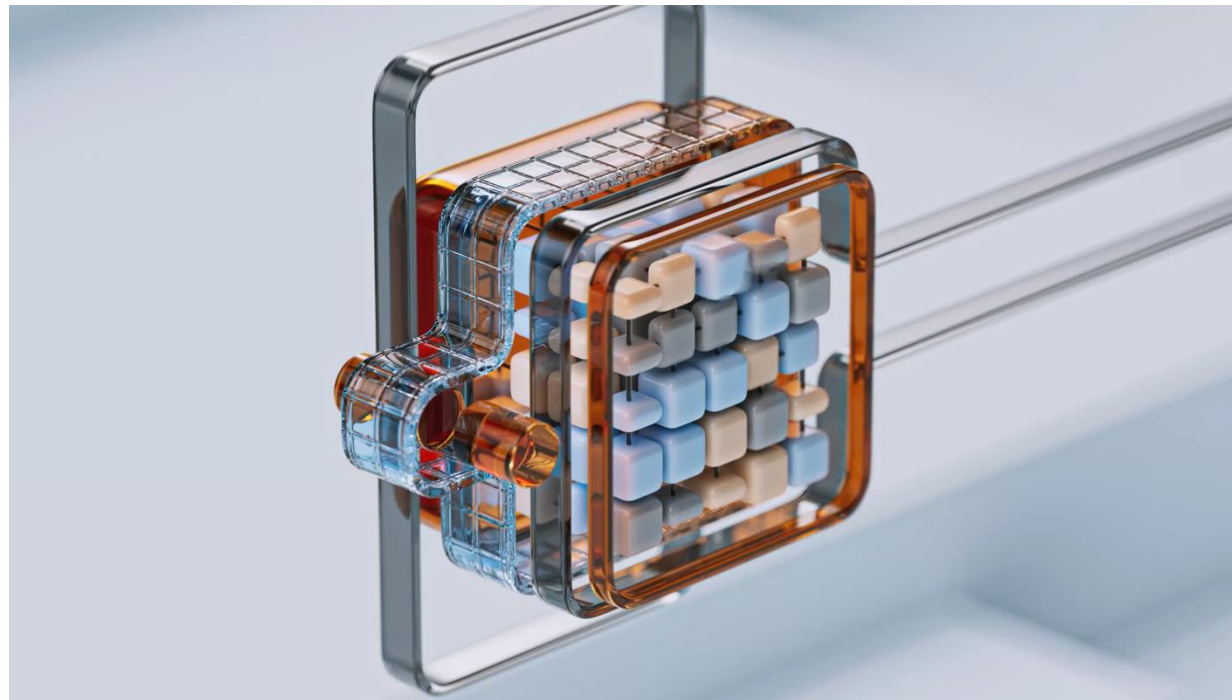


16-831: Introduction to Robot Learning, Fall 2024

Machine Learning & Deep Learning Refresher

Visualization of transformer, Google DeepMind



Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Logistics

- ❑ HW1 will be released tonight

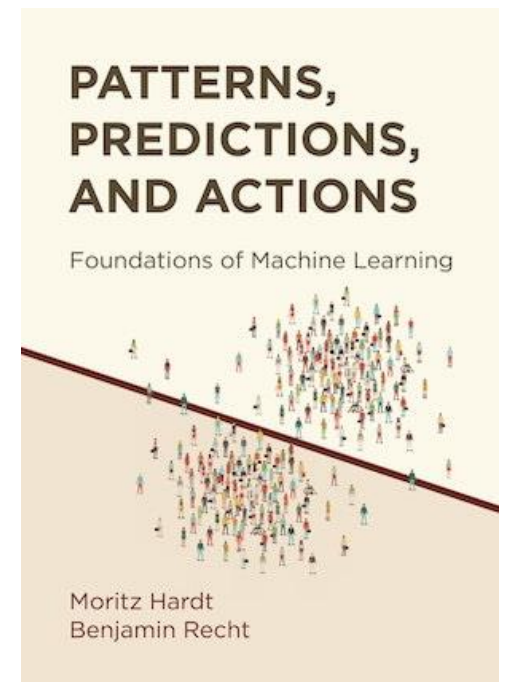
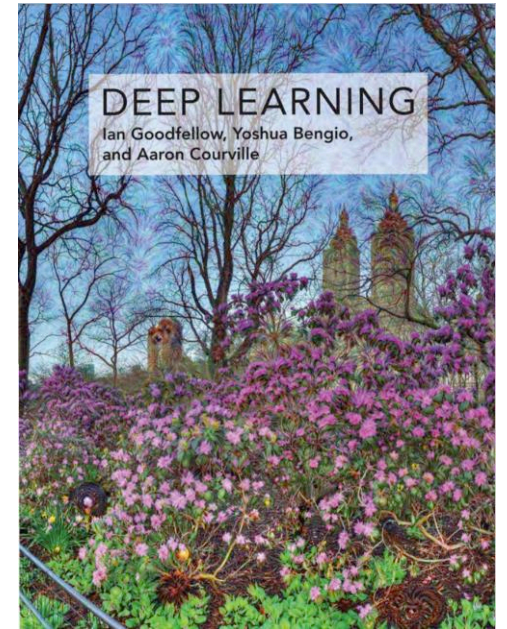
Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☒ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☐ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Before We Start ...

- ❑ Some slides are from Caltech CS155, Stanford CS231n, CMU 10-703
- ❑ Much of this field is in rapid motion!
- ❑ We all need to keep learning
- ❑ Many exciting and impactful ideas are inspired by other fields
- ❑ No standard textbook, but some useful resource:
 - Deep Learning Book (<https://www.deeplearningbook.org/>)
 - Patterns, Predictions, and Actions: Foundations of ML
 - PyTorch tutorial: <https://pytorch.org/tutorials/>
 - Other materials on Piazza and course website



Before We Start ...

- ❑ Many different perspectives
- ❑ Many exciting and impactful ideas are inspired by other fields
- ❑ Suggestions: don't get confused by fancy names – ask yourself what exactly makes this algorithm/method work so well in some domain



Yi Ma

@YiMaTweets

...

Mathematics is the art of giving the same name to different things(Henri Poincaré). Machine learning is the art of giving different names to the same thing.

Before We Start ...

- ❑ Anything that does not work today could work nicely in the future
- ❑ And often, big gaps between theory and practice
- ❑ Example 1: Ilya Sutskever was trying to convince everyone on the table to research on DNNs in 2008. Everyone knew DNNs didn't work at that time.



Before We Start ...

- ❑ Example 2: Overparameterization had been a bad idea in learning theory (and empirically).

Classification in a Probabilistic Setting

Theorem (Vapnik and Chervonenkis, 1971)

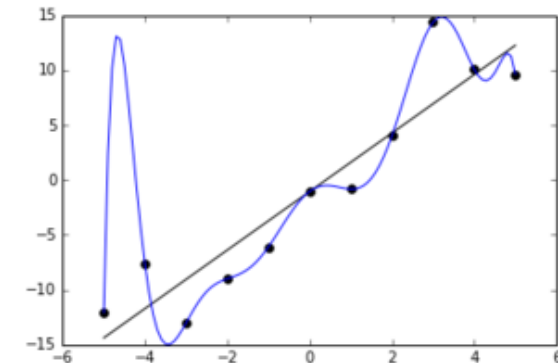
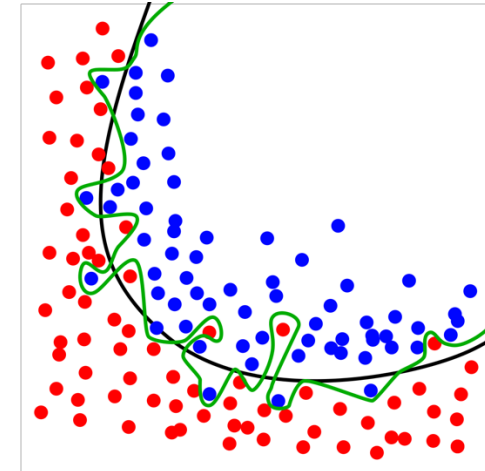
Consider $\mathcal{F} \subseteq \{-1, 1\}^{\mathcal{X}}$, $\mathcal{Y} = \{\pm 1\}$, $\ell = \ell_{01}$.
For every prob distribution P on $\mathcal{X} \times \{\pm 1\}$,
with probability $1 - \delta$ over n iid examples $(x_1, y_1), \dots, (x_n, y_n)$,
every f in $\mathcal{F}$ satisfies

$$\mathbb{E} \ell_f \leq \hat{\mathbb{E}} \ell_f + O\left(\sqrt{\frac{d_{VC}(\mathcal{F})}{n}}\right).$$

- For neural networks, VC-dimension:
 - increases **linearly** with number of parameters
 - depends on nonlinearity and depth
- For uniform bounds (that is, for all distributions and all $f \in \mathcal{F}$, proportions are close to probabilities; also for near-optimal prediction with $f \in \mathcal{F}$), this inequality is **tight** within a constant factor.

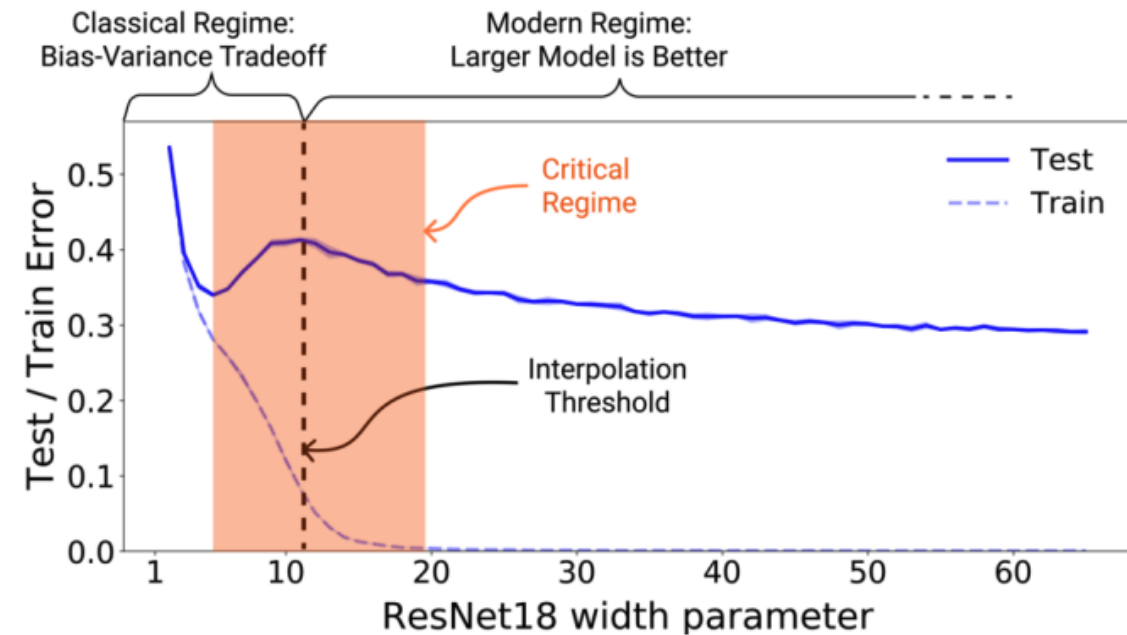
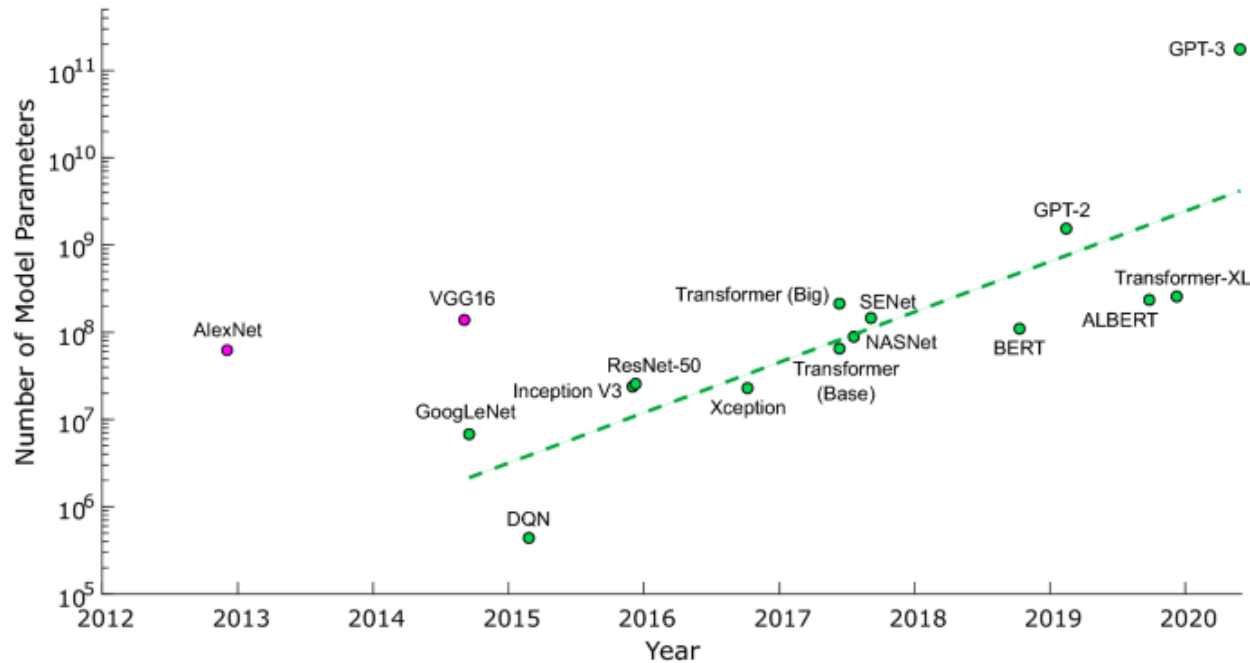
need $n \gg p$?

from Peter Bartlett's talk at NeurIPS'21



Before We Start ...

- Example 2: In reality, overparameterized models work better and better!
 - There are some good theoretical explanations now (e.g., implicit regularization of SGD)



Deep double descent: where bigger models and more data hurt*

Outline

❑ Supervised learning basics

- What is supervised learning?
- Model/architecture, training data, loss, learning objective, optimization
- Example: linear regression
- Generalization, overfitting, underfitting, validation set, model selection
- Probabilistic approach to supervised learning

❑ Deep learning

- Why deep learning?
- Fully connected deep neural networks (DNNs)
- How to train DNNs

❑ Next time: deep learning models (CNN, RNN/LSTM, transformer)

❑ Next time: unsupervised learning, generative model

❑ Next time: uncertainty quantification for DNNs, verifications of DNNs

Outline

☐ Supervised learning basics

- What is supervised learning?
- Model/architecture, training data, loss, learning objective, optimization
- Example: linear regression
- Generalization, overfitting, underfitting, validation set, model selection
- Probabilistic approach to supervised learning

☐ Deep learning

- Why deep learning?
- Fully connected deep neural networks (DNNs)
- How to train DNNs

☐ Next time: deep learning models (CNN, RNN/LSTM, transformer)

☐ Next time: unsupervised learning, generative model

☐ Next time: uncertainty quantification for DNNs, verifications of DNNs

The Basic Idea of Supervised Learning

- ❑ Learn a function $f: X \rightarrow Y$ from an input space X (observations) to an output space Y (targets), using a set of labeled examples $D_{\text{train}} = (x_1, y_1), \dots, (x_N, y_N)$
- ❑ “Generalization”: Ideally, such learned function f will perform well on the test data (not included in D)
- ❑ Example: Image classification

Supervised Learning



Input x

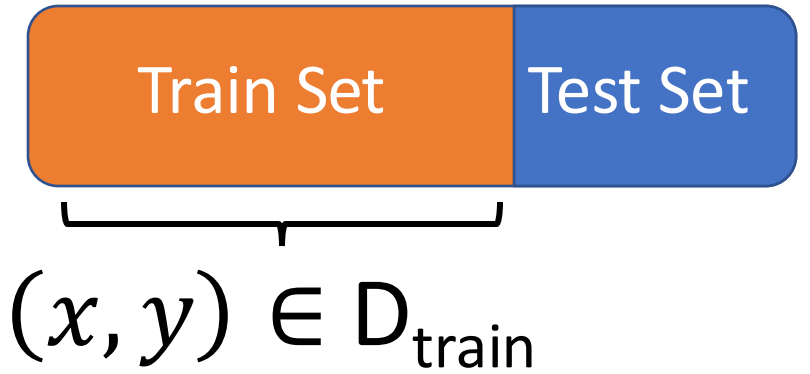


Model $f_{\theta}()$



“Labrador”

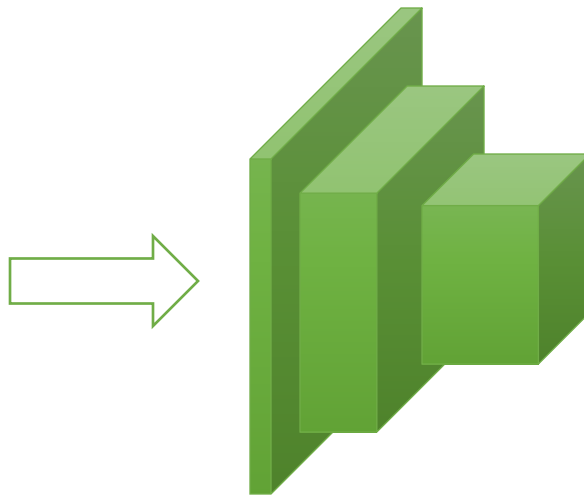
Output
 $y = f_{\theta}(x)$



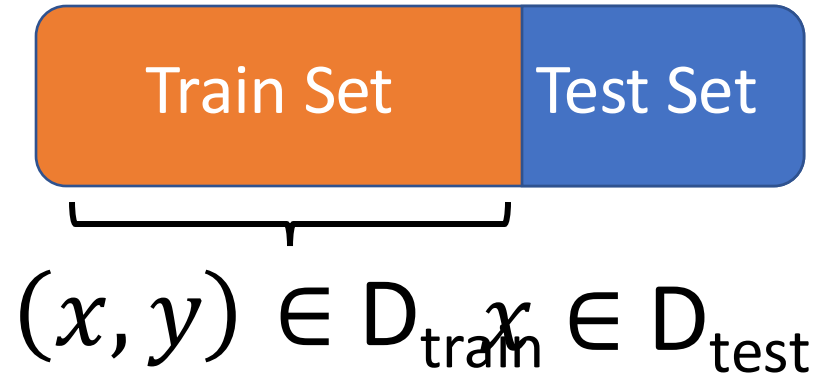
Supervised Learning



Input x



Model $f_{\theta}()$



“Labrador”

Output
 $y = f_{\theta}(x)$

Why Is It Hard?

- ❑ Learn a function $f: X \rightarrow Y$ from an input space X (observations) to an output space Y (targets), using a set of labeled examples $D_{\text{train}} = (x_1, y_1), \dots, (x_N, y_N)$
- ❑ “Generalization”: Ideally, such learned function f will perform well on the test data (not included in D)
- ❑ What we see and what a computer sees:

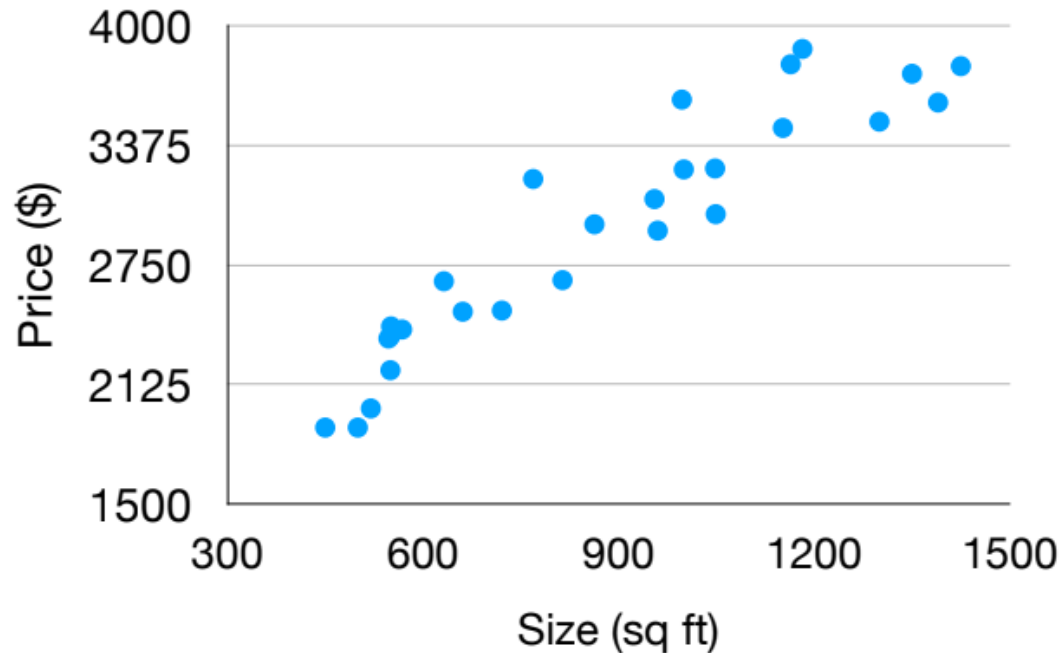


157	153	174	168	150	162	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	5	10	33	45	106	159	181
206	108	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	52	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	137	103	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	45	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	52	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	137	103	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

- ❑ It is why we need learning! To extract pattern from “complex” data.

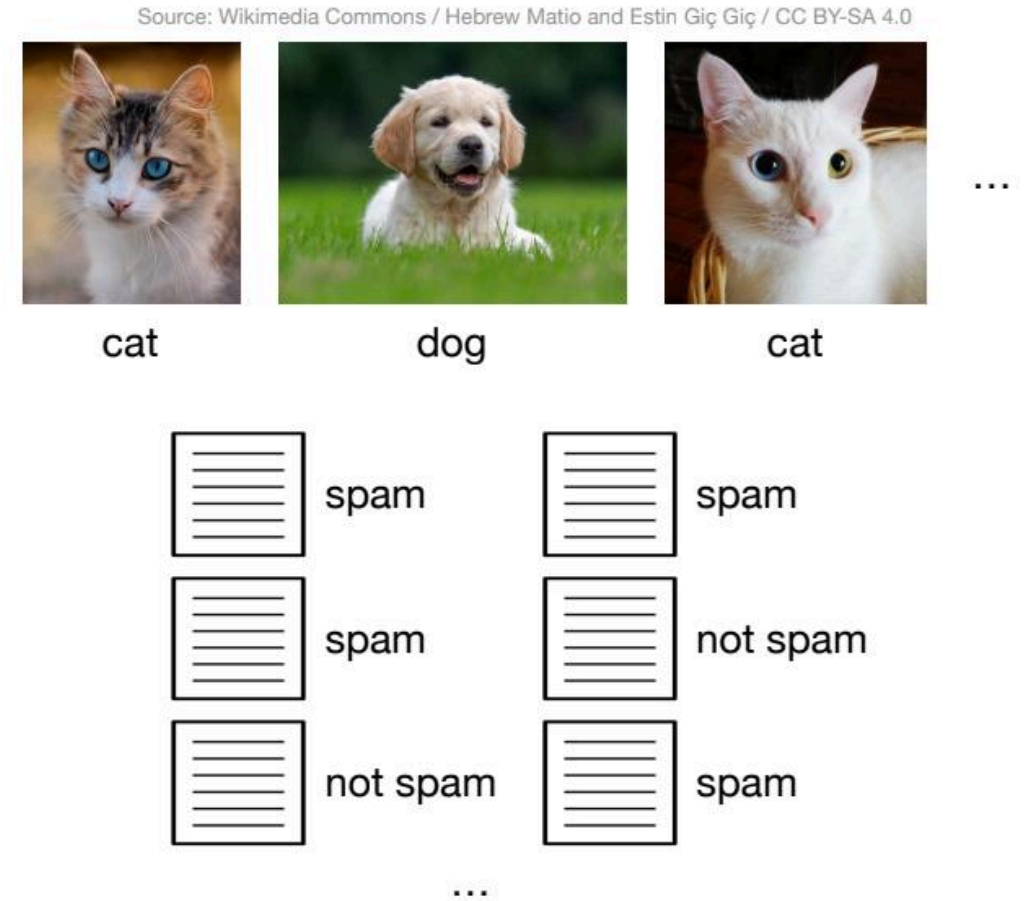
Types of Supervised Learning Problems



Regression: predict real values y

$Y = \mathbb{R}$ or $\mathbb{R}^n$

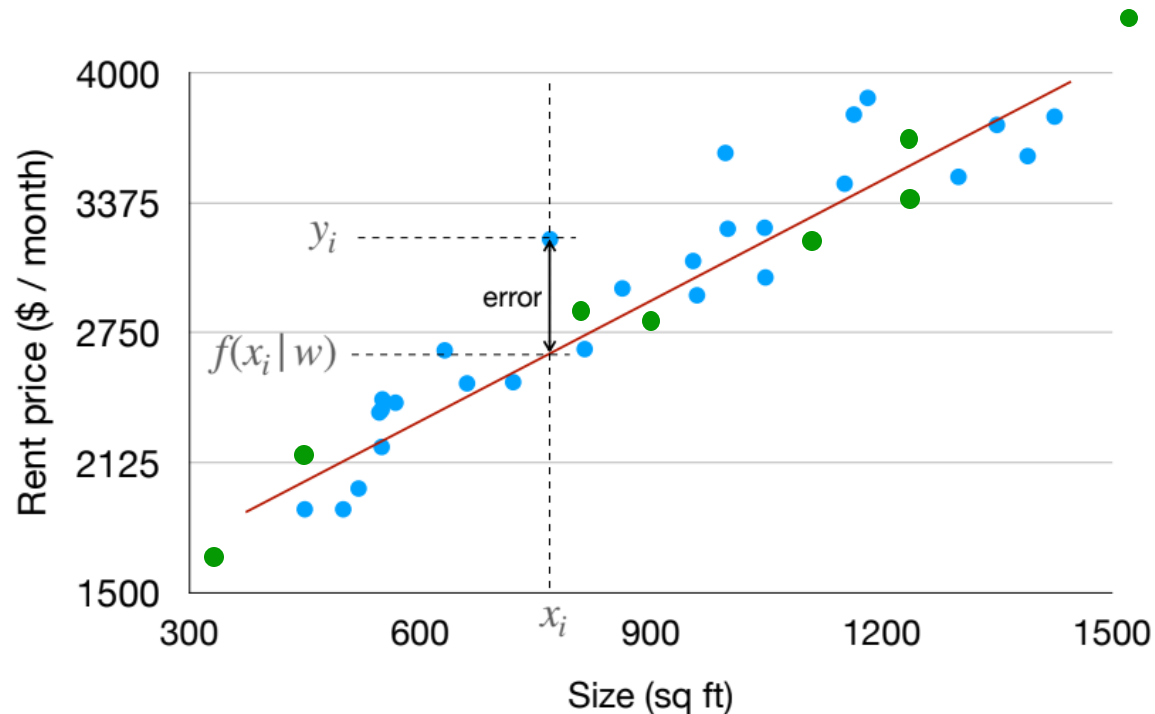
+ more... (e.g., object detection)



Classification: predict a class y from a fixed finite set

Example: Linear Regression

D_{train} : blue dots; D_{test} : green dots



❑ **Model:** Let's try to fit a straight line $f(x|w) = w^\top x$ (bias is included in w by introducing the “dummy feature” $x = [1; x]$)

❑ **Training goal:** $y_i \approx f(x_i|w)$

❑ **Loss:** squared loss $L(y, y') = (y - y')^2$

❑ **Learning objective:**

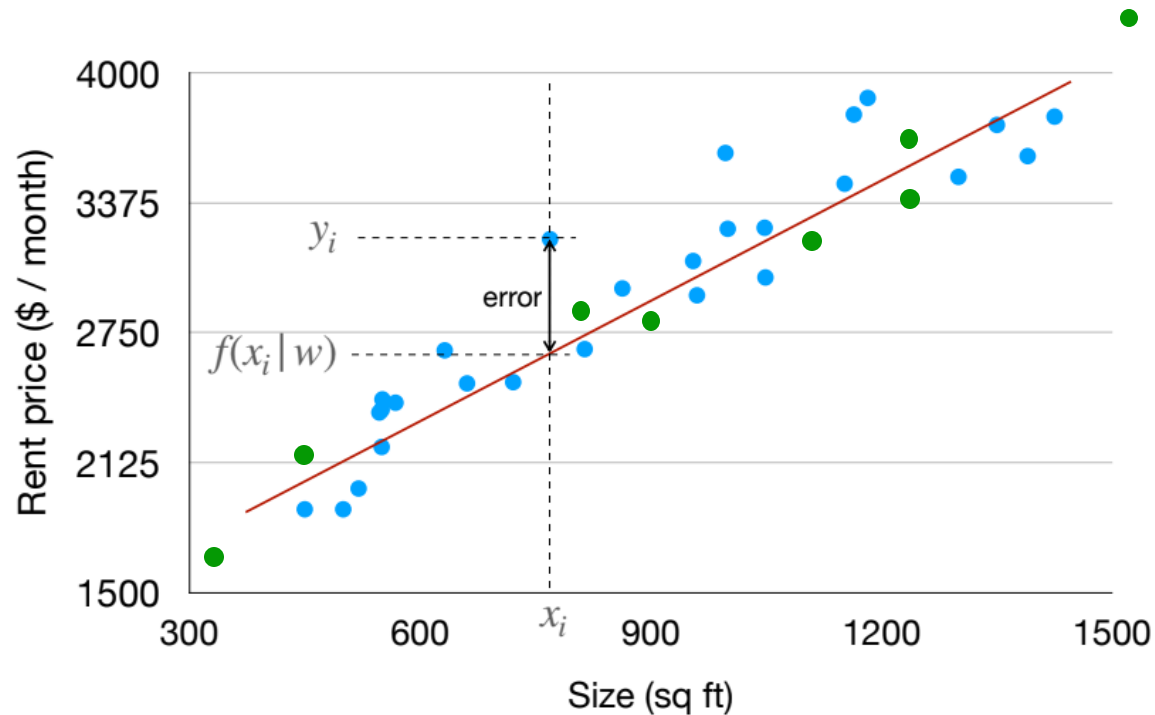
$$\operatorname{argmin}_w \sum_{i \in D_{\text{train}}} L(y_i, f(x_i|w))$$

❑ **Generalization:** $L(y, f(x|w))$ on the green dots (D_{test}) is small

- Interpolation v.s. extrapolation
- Domain shift: D_{train} and D_{test} are from different distributions

Example: Linear Regression

D_{train} : blue dots; D_{test} : green dots



❑ **Loss:** squared loss $L(y, y') = (y - y')^2$

❑ **Learning objective:**

$$\operatorname{argmin}_w L_N(w) = \sum_{i \in D_{\text{train}}} L(y_i, f(x_i|w))$$

❑ **Overfitting:** test error $\gg$ training error

❑ **Underfitting:** test and training error are similar and both high

❑ **Optimization:** optimize the learning objective

- Closed-form solution $w = (X^T X)^{-1} X^T Y$
- Gradient descent: $w \leftarrow w - \eta \nabla_w L_N(w)$
- SGD:

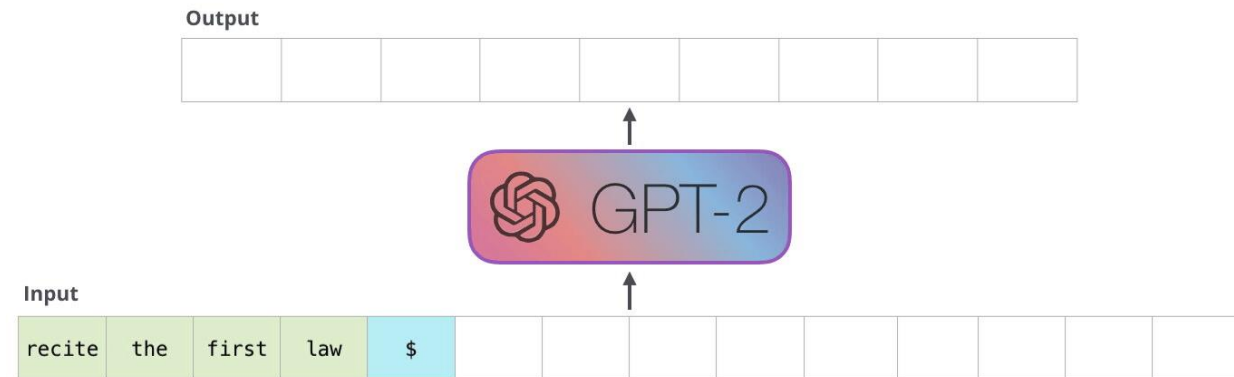
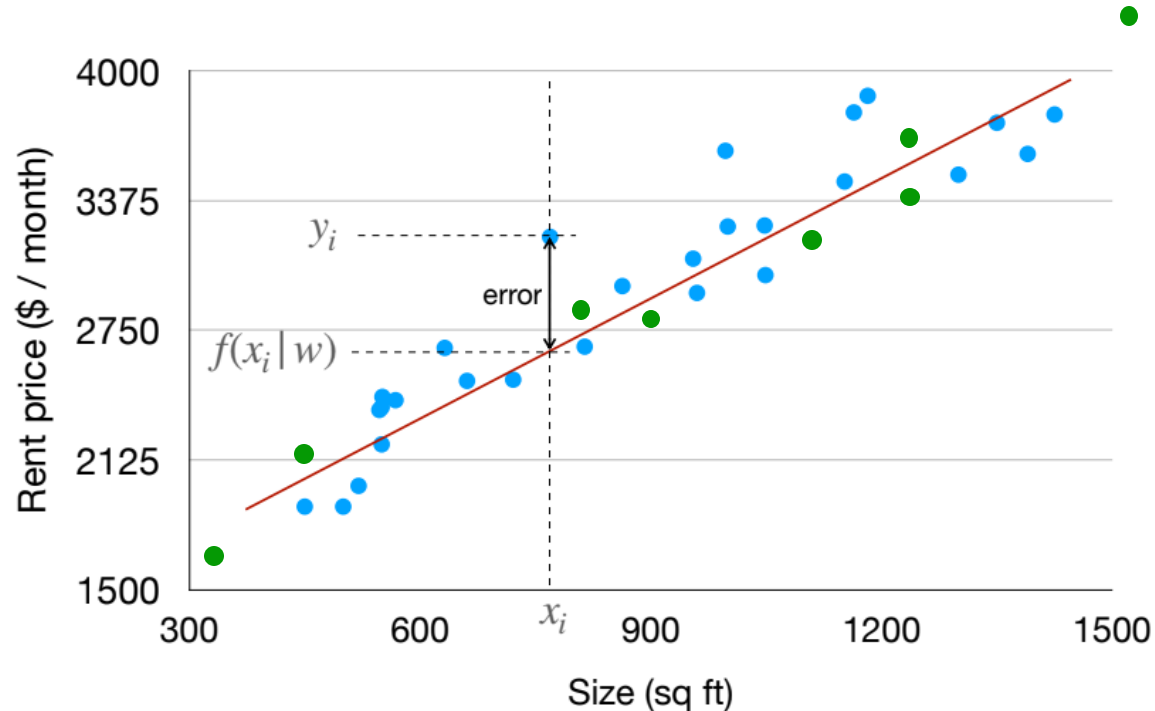
$$w \leftarrow w - \eta \nabla_w \sum_{i \in \text{Batch}} L(y_i, f(x_i|w))$$

Not Just Linear Regression!

Machine learning foundations (no matter what model you are using):

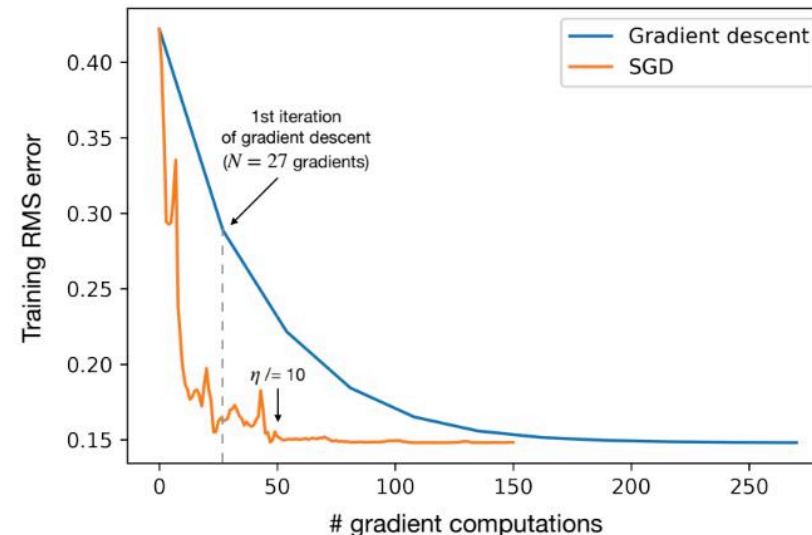
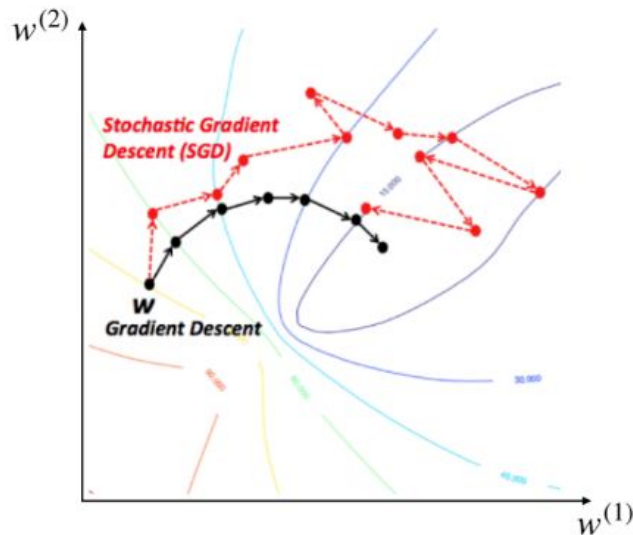
❑ **Model/architecture, training data, loss, learning objective, generalization, overfitting, underfitting, optimization**

D_{train} : blue dots; D_{test} : green dots



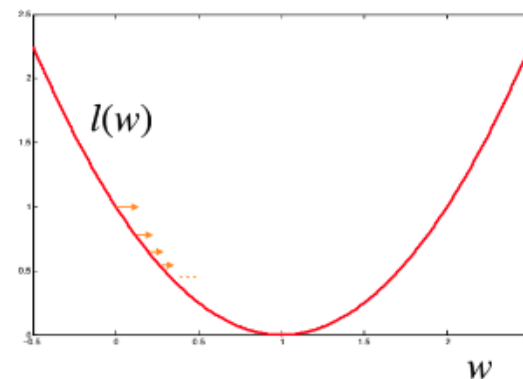
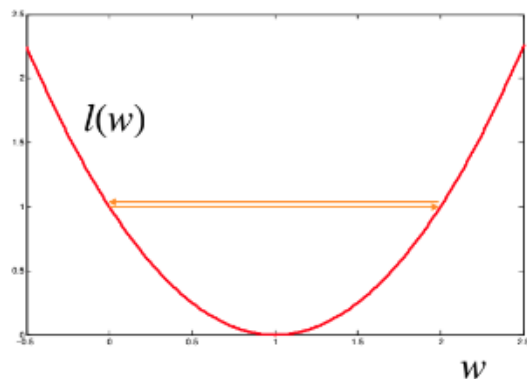
Optimization: GD v.s. SGD

- ❑ **Learning objective:** $\operatorname{argmin}_w L_N(w) = \sum_{i \in D_{\text{train}}} L(y_i, f(x_i|w))$
- ❑ **Gradient descent (GD):** $w \leftarrow w - \eta \nabla_w L_N(w)$
 - Prohibitively expensive if the training dataset is large!
- ❑ **Mini-batch stochastic gradient descent (SGD):** $w \leftarrow w - \eta \nabla_w \sum_{i \in \text{Batch}} L(y_i, f(x_i|w))$
 - An online optimization algorithm! (only needs access to one batch at a time)
 - Leverages fast vector operations (especially on GPU)
 - Can be parallelized (e.g., different cores compute gradients on different mini-batches)
 - Not just “DL”: useful even for least-squares regression on large datasets!



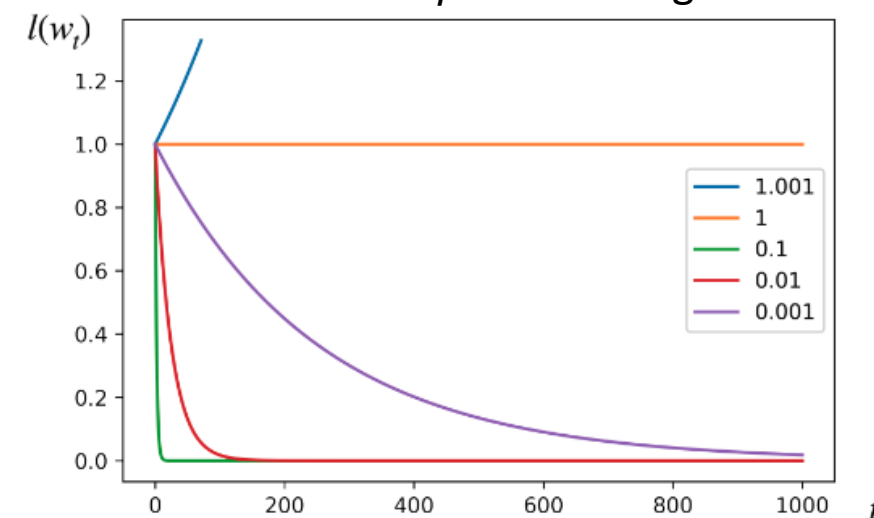
Optimization: Learning Rate

- ❑ If too large, the algorithm can oscillate or even diverge (left figure)!
- ❑ If too small, the algorithm can take very long to converge (right figure)!



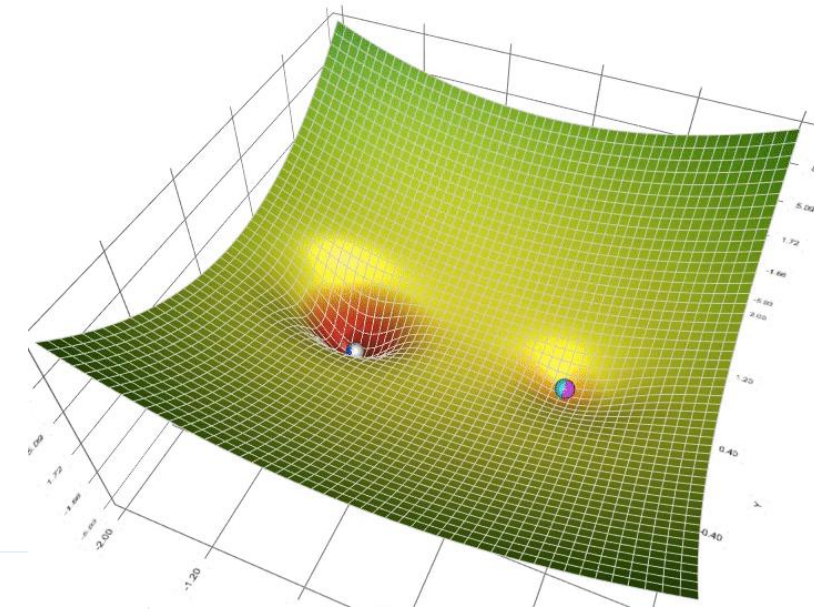
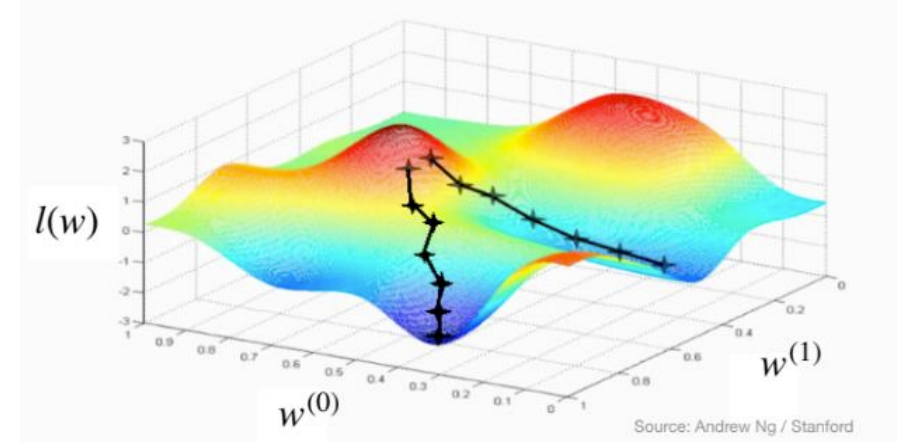
- ❑ Ideally, we want η as large as possible w/o diverging
- ❑ Practical rules of thumb:
 - Divide the loss by the # of examples (**normalize**)
 - Start with a large step size
 - Whenever the **validation error** stops going down, lower the step size, e.g., $\eta_{t+1} \leftarrow \eta_t/2$
 - Stop when the validation error makes no further progress (**early stopping**)

GD with different η in linear regression



Optimization: Beyond SGD

- ❑ Non-convex problems are hard to optimize
 - Multiple local minima
 - Good news: in deep learning, we don't really care about finding the global optimum
 - It is more important to find a good local optimum.
- ❑ Many optimization methods beyond SGD:
 - Momentum, AdaGrad, RMSProp, Adam, ...
 - Aim to improve the robustness and efficiency
 - Many trade-offs
 - Example: how momentum works



<https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c>

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \mathbf{v}_{t+1} \\ \mathbf{v}_{t+1} &= \rho \mathbf{v}_t - \eta \mathbf{g}(\mathbf{w}_t) \end{aligned}$$

Velocity

Momentum



Image 2: SGD without momentum



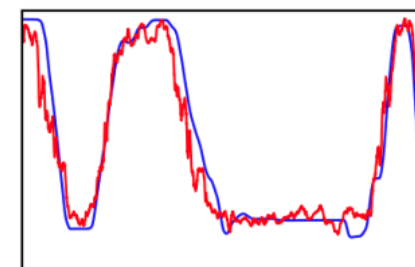
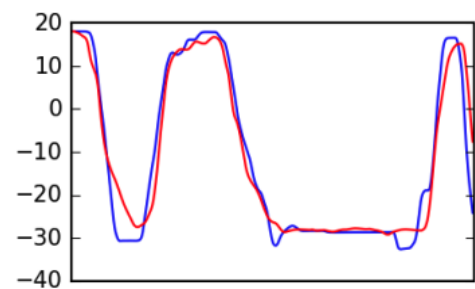
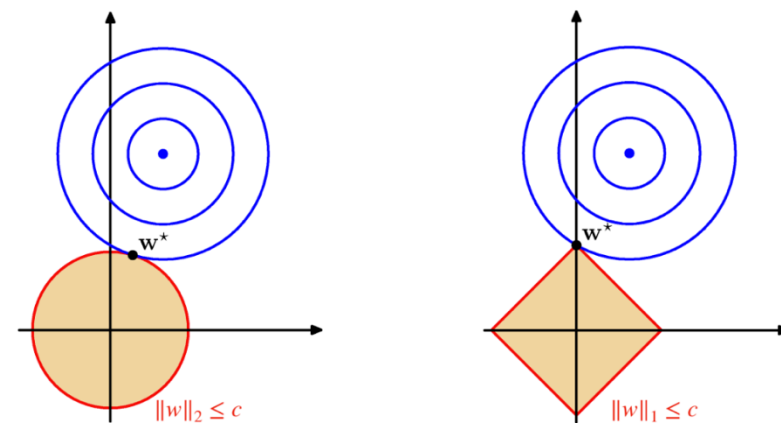
Image 3: SGD with momentum

Optimization: Regularization

- ❑ Add some “inductive bias” on w or $f(x|w)$
 - E.g., sparsity, smooth, etc.
- ❑ Two classic regularizations:
 - Ridge regression ($R(w) = \|w\|^2$) for minimum “energy”
 - Lasso ($R(w) = \|w\|_1$) for sparsity

$$\operatorname{argmin}_w \sum_{i \in D_{\text{train}}} L(y_i, f(x_i|w)) + \lambda \cdot R(w)$$

- ❑ Useful for robot learning!
 - Example: smooth dynamics learning and smooth imitation learning



Expected Test Error and Bias-Variance Decomposition

- Consider model parameter $w = w_D$ as a function of the training dataset D
- D consists of N i.i.d. samples from $P(x, y)$ (no domain shift)

$$w_D = \operatorname{argmin}_w \sum_{i \in D} L(y_i, f(x_i|w))$$

- Expected test error:

$$\mathbb{E}_D \mathbb{E}_{(x,y) \sim P(x,y)} [L(y, f(x|w_D))]$$

- **Statistical learning theory:** quantify $\hat{\epsilon}$ as a function of N , f , and P
- Bias-variance decomposition assuming squared loss:
 - $F(x) = \mathbb{E}_D [f(x|w_D)]$ is the average prediction of the model

$$\begin{aligned} & \mathbb{E}_D \mathbb{E}_{(x,y) \sim P(x,y)} [L(y, f(x|w_D))] \\ &= \mathbb{E}_{(x,y) \sim P(x,y)} \left[\underbrace{\mathbb{E}_D [(f(x|w_D) - F(x))^2]}_{\text{variance}} + \underbrace{(F(x) - y)^2}_{\text{bias}} \right] \end{aligned}$$

Bias-Variance Decomposition

□ Bias-variance decomposition assuming squared loss:

- $F(x) = \mathbb{E}_D[f(x|w_D)]$ is the average prediction of the model

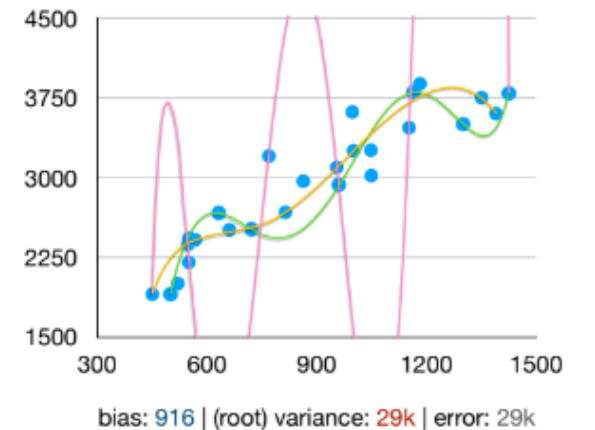
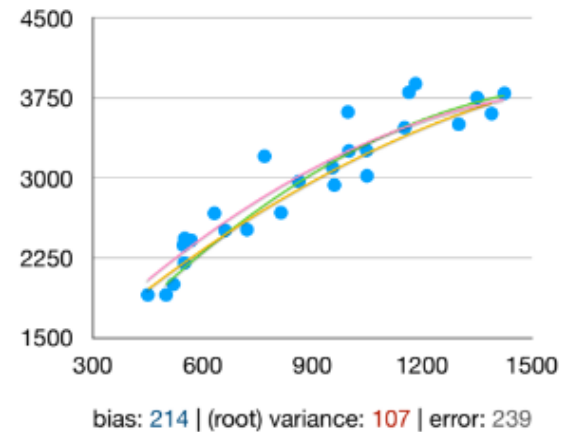
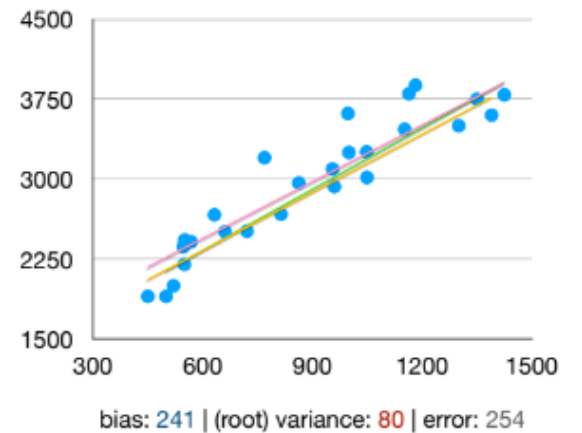
$$\begin{aligned} & \mathbb{E}_D \mathbb{E}_{(x,y) \sim P(x,y)} [L(y, f(x|w_D))] \\ = & \mathbb{E}_{(x,y) \sim P(x,y)} \left[\underbrace{\mathbb{E}_D [(f(x|w_D) - F(x))^2]}_{\text{variance}} + \underbrace{(F(x) - y)^2}_{\text{bias}} \right] \end{aligned}$$

□ Overfitting implies high variance

- Variance increases with model complexity
- Variance decreases with more training data

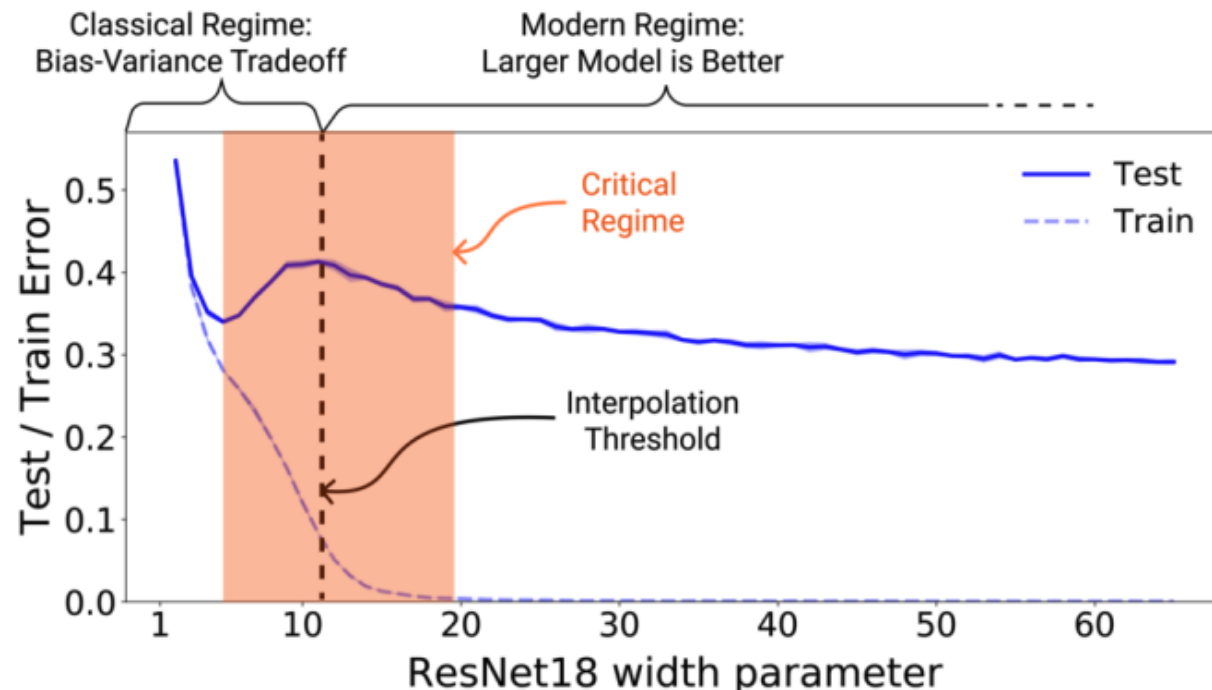
□ Underfitting implies high bias

- Happens when model complexity is too low
- Extreme case: $f(x|w_D) = \text{constant}$, no variance!



Bias-Variance Decomposition

- ❑ Overfitting implies high variance
 - Variance increases with model complexity
 - Variance decreases with more training data
- ❑ Underfitting implies high bias
 - Happens when model complexity is too low
- ❑ However, this tradeoff might not hold when the model is very large!



DEEP DOUBLE DESCENT: WHERE BIGGER MODELS AND MORE DATA HURT

Preetum Nakkiran*
Harvard University

Gal Kaplun[†]
Harvard University

Yamini Bansal[†]
Harvard University

Tristan Yang
Harvard University

Boaz Barak
Harvard University

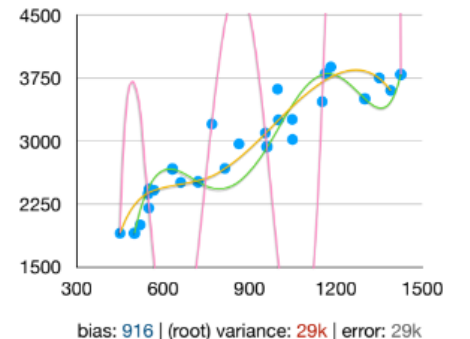
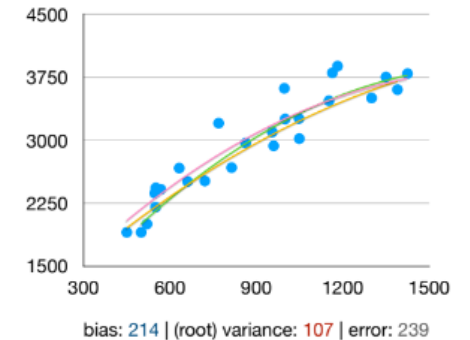
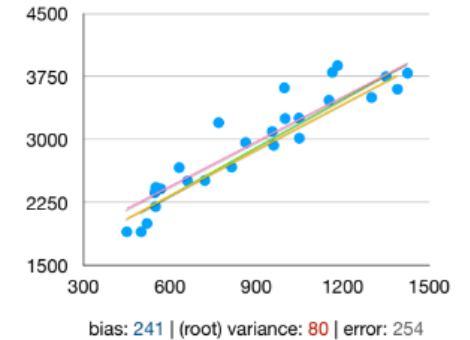
Ilya Sutskever
OpenAI

Model Selection and Validation Set

- ❑ How do we choose from these models and tune hyperparameters such as learning rates?
- ❑ We only have training dataset – cannot measure the true test error
- ❑ The key principle of machine learning: “Test and train conditions must match” (Vinyals et al., Matching Networks for One-Shot Learning)



- ❑ Split the original dataset into a **training** set and a **validation** set
- ❑ Train model on the training set
- ❑ Evaluate on the validation set to *estimate* the test error
- ❑ Can also do this procedure in a k-fold way (split the original data into k equal parts)



Probabilistic Approach to Supervised Learning

□ So far, the idea is: find a parameter w such that $y_i \approx f(x_i|w)$ for $(x_i, y_i) \in D_{\text{train}}$

□ Probabilistic perspective:

- Model a probability dist. $P(y|x; w)$ of label y given inputs x
- Choose a “**model**” (form) for $P(y|x; w)$
- Write the likelihood of w , i.e., the probability of observing the labels y_i of the training dataset D_{train} given the inputs x_i :

$$P(D_{\text{train}}|w) = \prod_i P(y_i|x_i; w)$$

- Learning objective (**maximum likelihood estimation, MLE**): find w that maximizes the log likelihood:

$$\operatorname{argmin}_w \log P(D_{\text{train}}|w) = \sum_i \log P(y_i|x_i; w) = -L(w)$$

□ Bayesian perspective: “weight” this likelihood by prior

$$P(w|D_{\text{train}}) \propto P(D_{\text{train}}|w)P(w)$$

Example: Logistic Regression

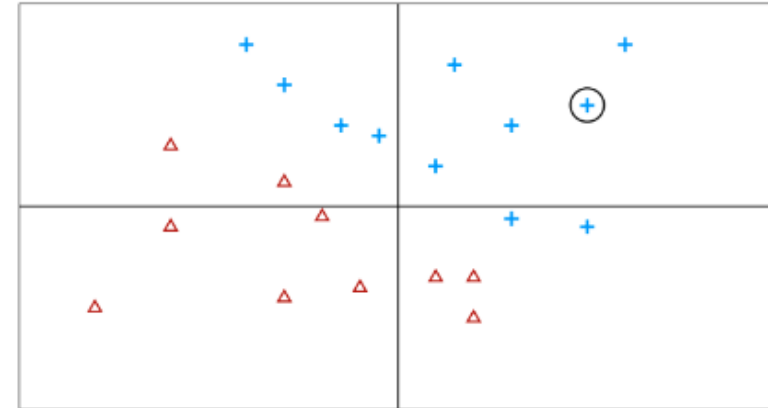
- ❑ Binary classification: prediction is $\text{sign}(f(x|w)) \in \{+1, -1\}$
- ❑ Idea: we want the **raw score** $f(x|w)$ to model the prob. of each class: $\sigma(f(x|w)) \approx \text{prob. that } y = +1$
 - σ : logistic/sigmoid function
 - Very useful property: $\sigma'(a) = (1 - \sigma(a))\sigma(a)$

- ❑ Maximize log likelihood of w on the training dataset:

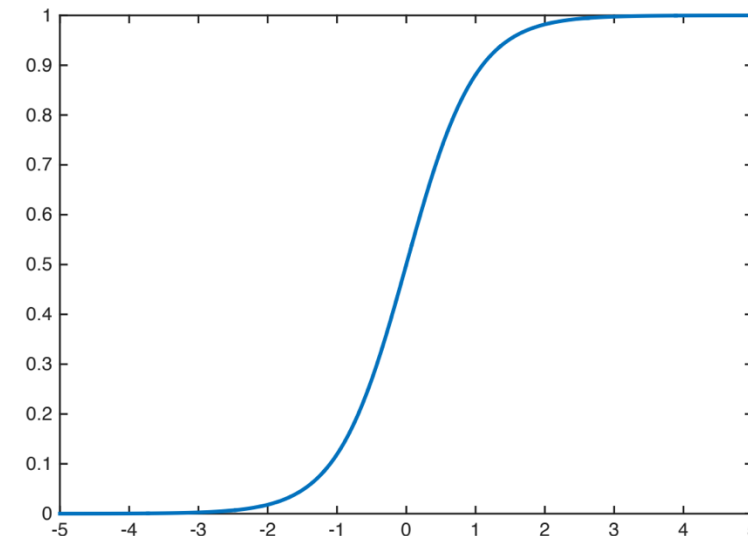
$$\log P(D_{\text{train}}|w) = \sum_i \underbrace{1_{\{y_i=+1\}} \log \sigma(f(x_i|w)) + 1_{\{y_i=-1\}} \log(1 - \sigma(f(x_i|w)))}_{\text{(minus) logistic loss}}$$

- A.k.a. log loss, binary cross-entropy loss
- Gradient: $-(1_{\{y_i=+1\}} - \sigma(f(x_i|w))) \cdot \nabla f(x_i|w)$

$\downarrow$ $\downarrow$
true probability that $y_i = +1$ model's prediction



$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

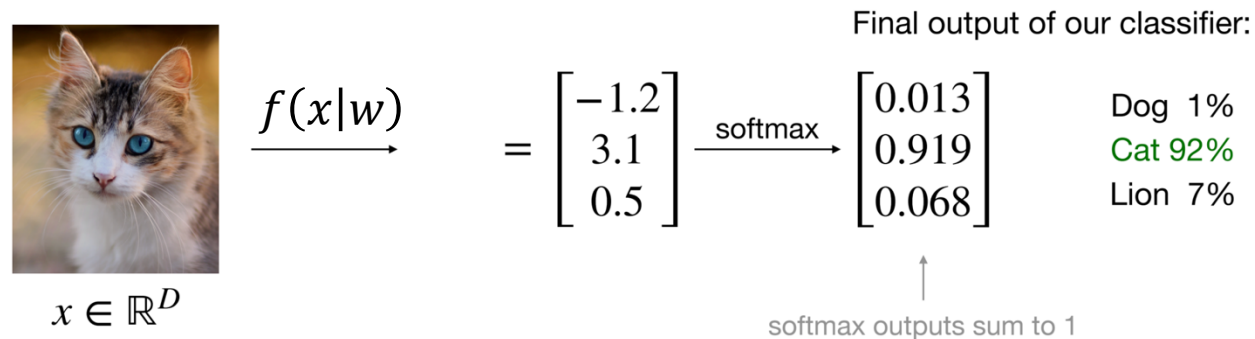


Multiclass Logistic Regression

□ The sigmoid is replaced by the softmax:

$$\text{softmax} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_K \end{bmatrix} = \frac{1}{\sum_{k=1}^K \exp(a_k)} \begin{bmatrix} \exp(a_1) \\ \exp(a_2) \\ \vdots \\ \exp(a_K) \end{bmatrix}$$

□ Example: $K = 3$, $Y = \{\text{dog, cat, lion}\}$



□ Cross-entropy loss (logistic loss is the special case when $K = 2$):

$$\sum_i \sum_{k=1}^K 1_{\{y_i=k\}} \log \text{softmax}(f(x_i|w))_k$$

Summary: Supervised Learning Pipeline

Linear regression as an example:

- ❑ **Training dataset:** $D_{\text{train}} = (x_1, y_1), \dots, (x_N, y_N)$
- ❑ **Model:** $f(x|w) = w^\top x$
- ❑ **Loss function:** $L(y, y') = (y - y')^2$
- ❑ **Learning objective:** $\operatorname{argmin}_w \sum_{i \in D_{\text{train}}} L(y_i, f(x_i|w))$
- ❑ **Optimization:** GD, SGD, ...
 - Learning rate is important
- ❑ **Cross validation & model selection**
 - Goal: Generalization w/o overfitting or underfitting
- ❑ **Probabilistic approach to supervised learning**
 - Example: logistic regression

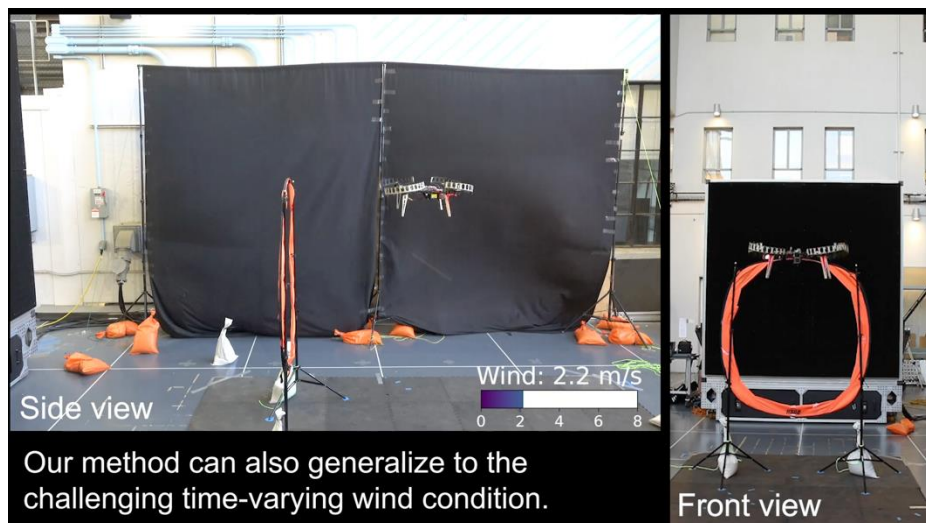
Supervised Learning in Robotics

- ❑ Training dataset: $D_{\text{train}} = (x_1, y_1), \dots, (x_N, y_N)$
- ❑ y can be many things!
 - Dynamics (model-based RL and control)
 - Action (imitation learning)
- ❑ How to generate y and how to use it are more important!



Residual
force prediction

$$\dot{x} = f_n(x, u) + \underset{\text{unknown}}{f(x, u)}$$



Outline

☐ Supervised learning basics

- What is supervised learning?
- Model/architecture, training data, loss, learning objective, optimization
- Example: linear regression
- Generalization, overfitting, underfitting, validation set, model selection
- Probabilistic approach to supervised learning

☐ Deep learning

- Why deep learning?
- Fully connected deep neural networks (DNNs)
- How to train DNNs

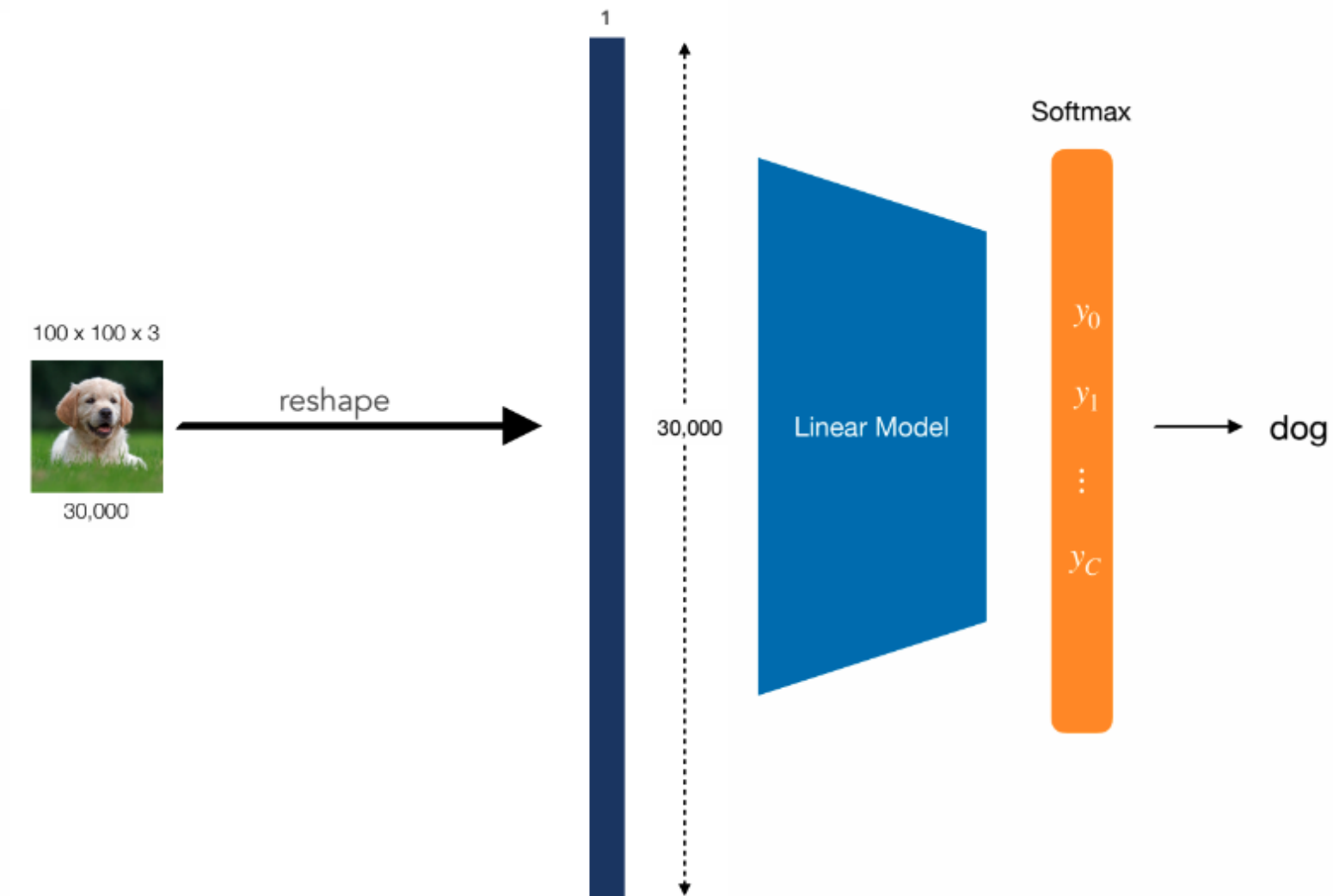
☐ Next time: deep learning models (CNN, RNN/LSTM, transformer)

☐ Next time: unsupervised learning, generative model

☐ Next time: uncertainty quantification for DNNs, verifications of DNNs

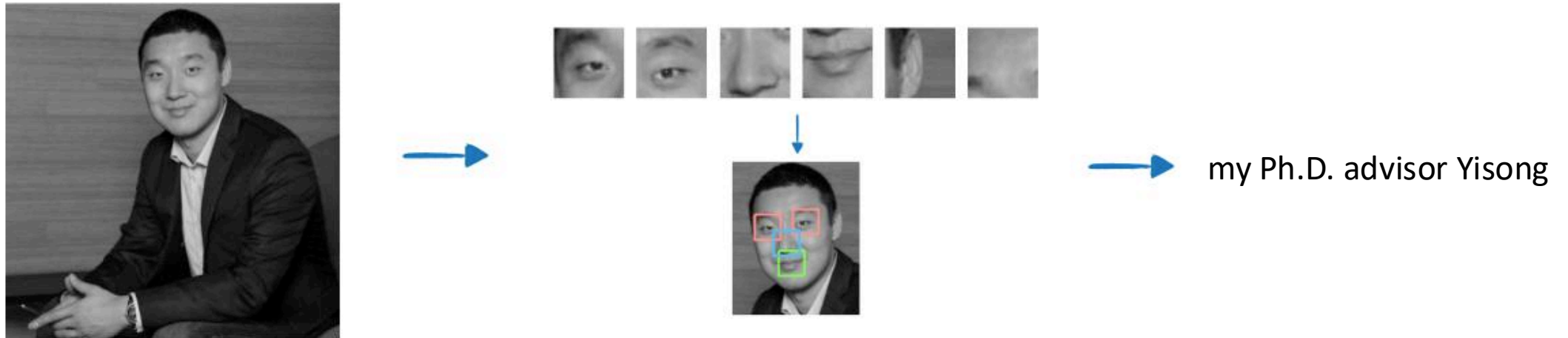
Deep Learning: Why?

Linear logistic regression for image classification: Does it work?



Deep Learning: Why?

- ❑ Problem: A linear model ($f(x|w) = w^\top x$ or $w^\top \phi(x)$) considers each feature **independently** and regresses the weight w with which it contributes to the label.
- ❑ But in images, individual features (pixels) are not meaningful. What matters is the relation between pixels!
 - Example: to recognize a person we need to look at **parts and relation between parts**

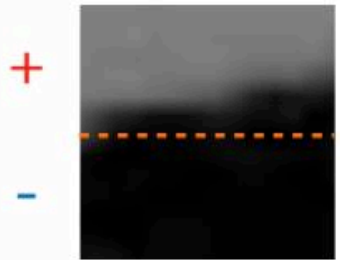


- ❑ In theory we could write an **embedding** $\phi(x)$ that encodes all these features. But can't we learn it?

The “Artificial” Neuron Model

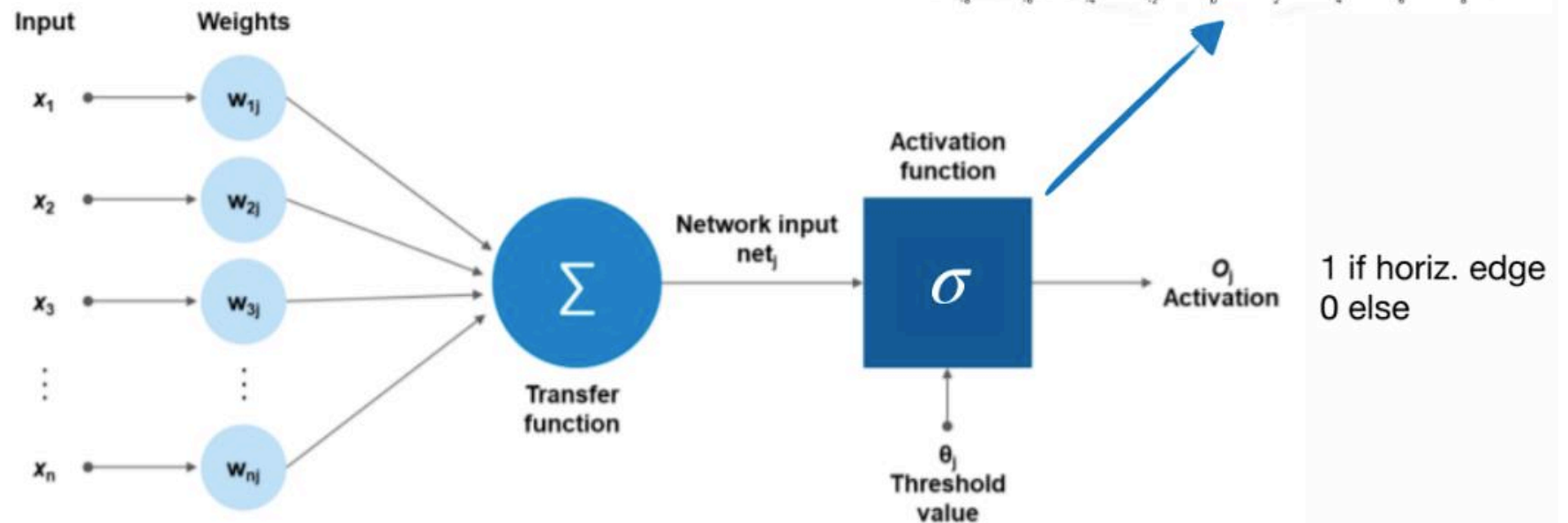
❑ Replicate biological neurons in code

Upper inputs have positive weights

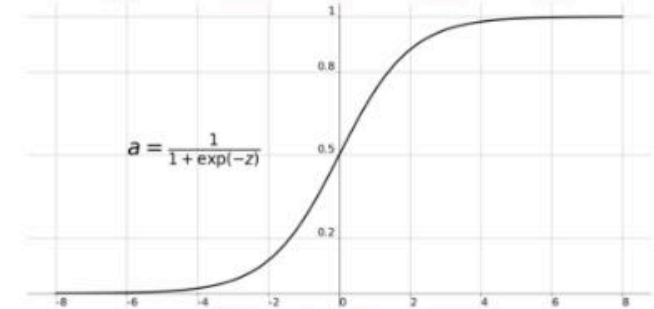


Bottom inputs have negative weights

Photoreceptors

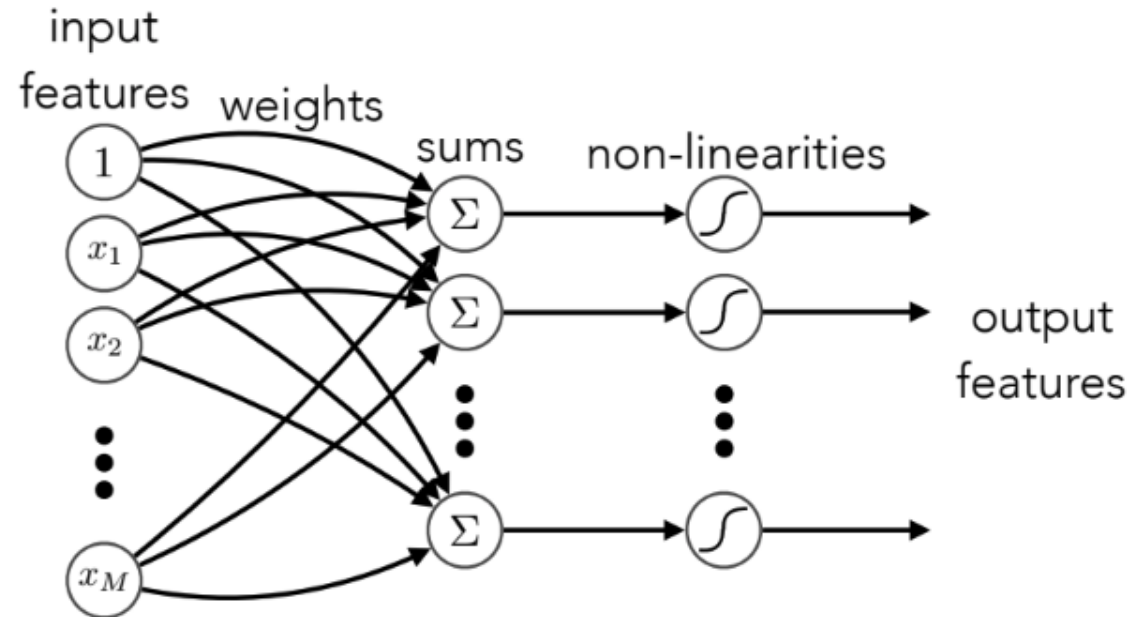


Sigmoid Function

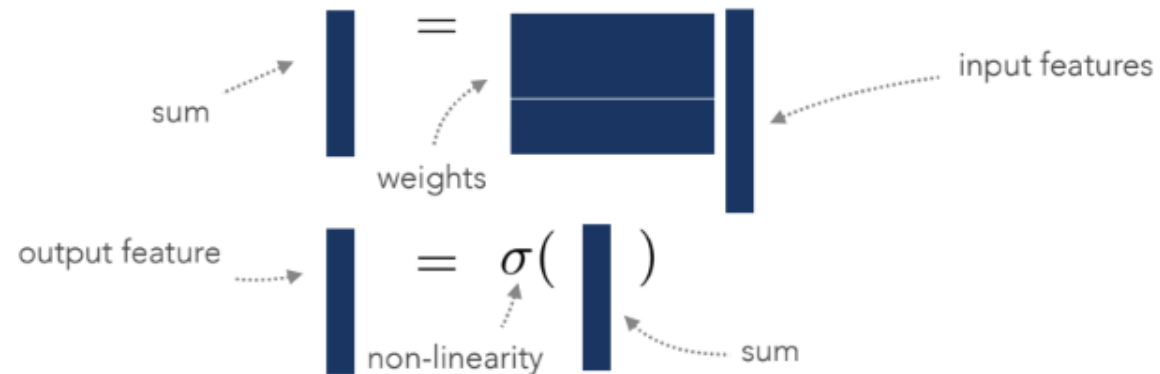


Deep Neural Network

□ One “layer”:

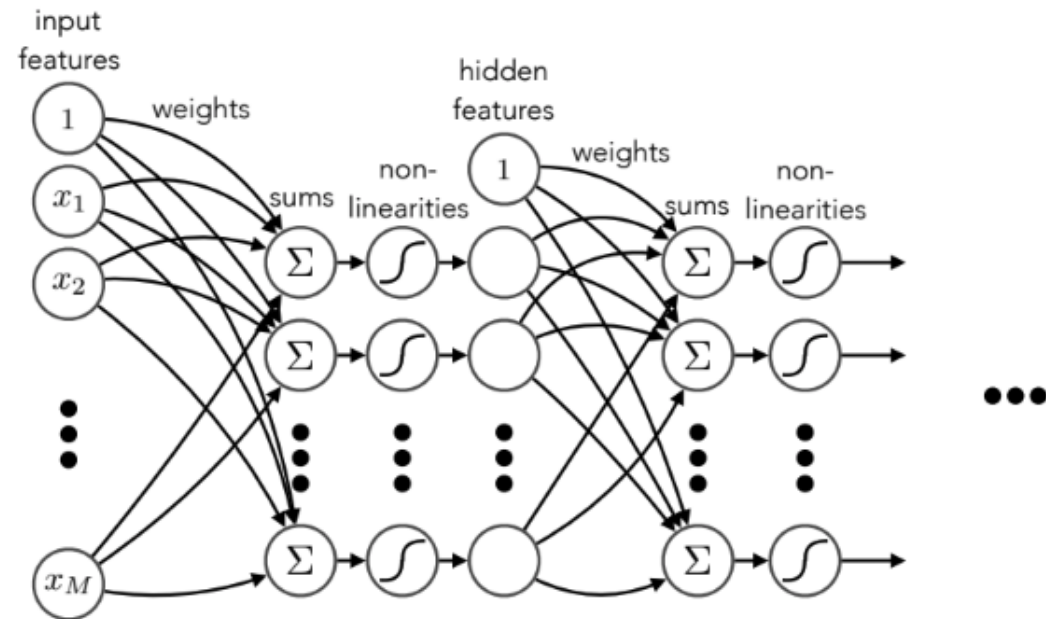


layer: *parallelized weighted sum and non-linearity*



Deep Neural Network

□ Multi-layer:

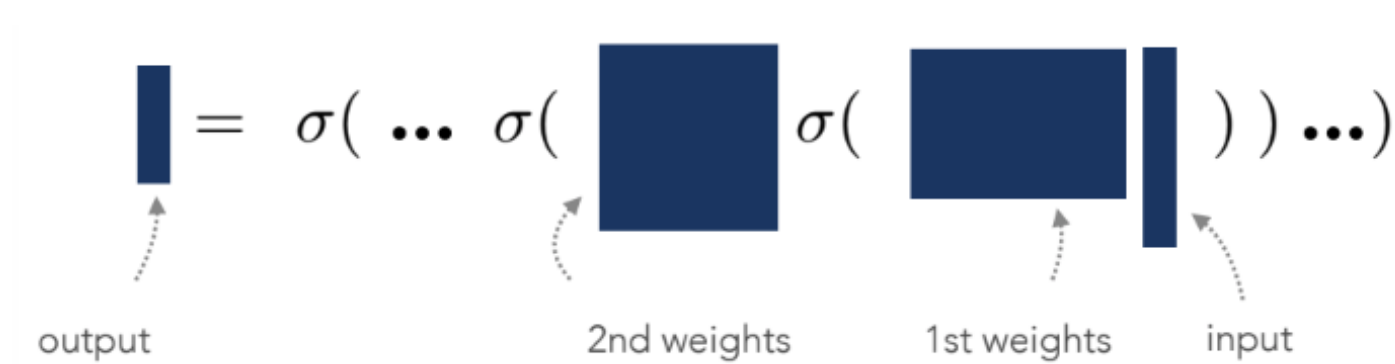


network: sequence of parallelized weighted sums and non-linearities

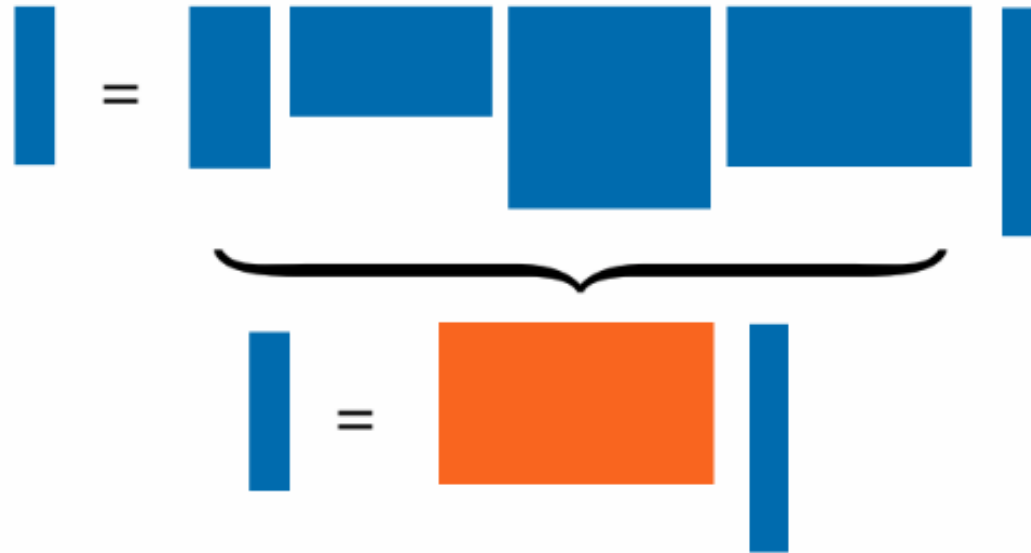
$$\text{output} = \sigma(\dots \sigma(\text{2nd weights} \sigma(\text{1st weights} \text{input})) \dots)$$

The equation shows the sequence of operations in a deep neural network. The input is a vertical bar, followed by a summation and non-linearity operation (σ) with '1st weights' (a horizontal bar). This is followed by another summation and non-linearity operation (σ) with '2nd weights' (a square). The sequence continues with more summations and non-linearities, eventually leading to the 'output' (a vertical bar).

The Role of Nonlinearities

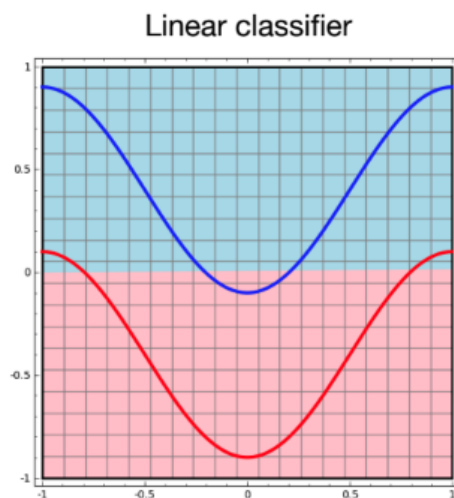
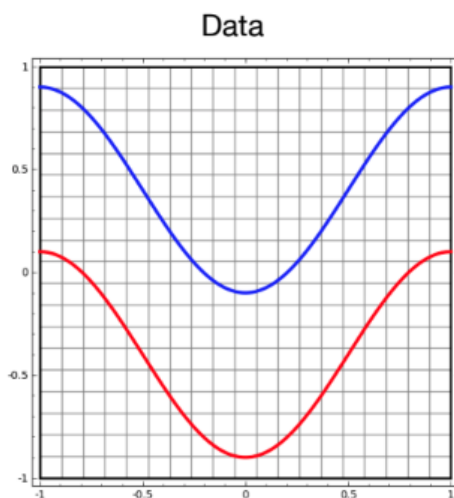
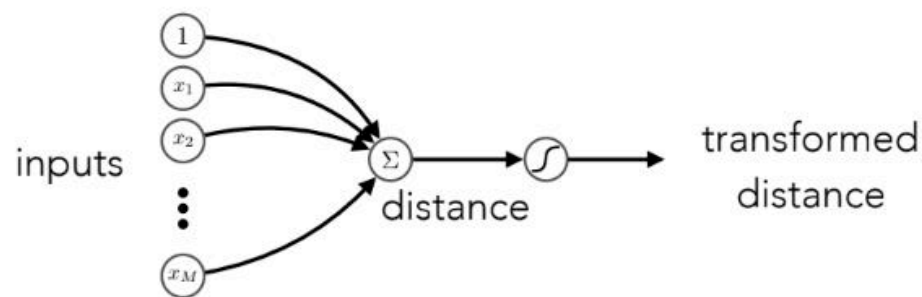


□ If we didn't have non-linearities, the whole network would reduce to a linear function!

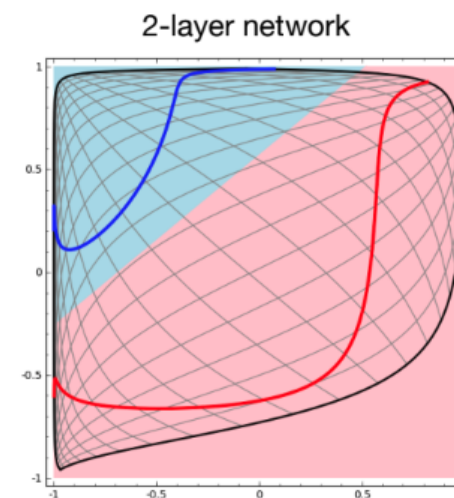


The Role of Nonlinearities: Geometry Perspective

- ❑ The linear part $w^T x$ defines a hyperplane
- ❑ The non-linearity “transforms” this distance:



$$y = \text{softmax}(w \cdot x + b)$$

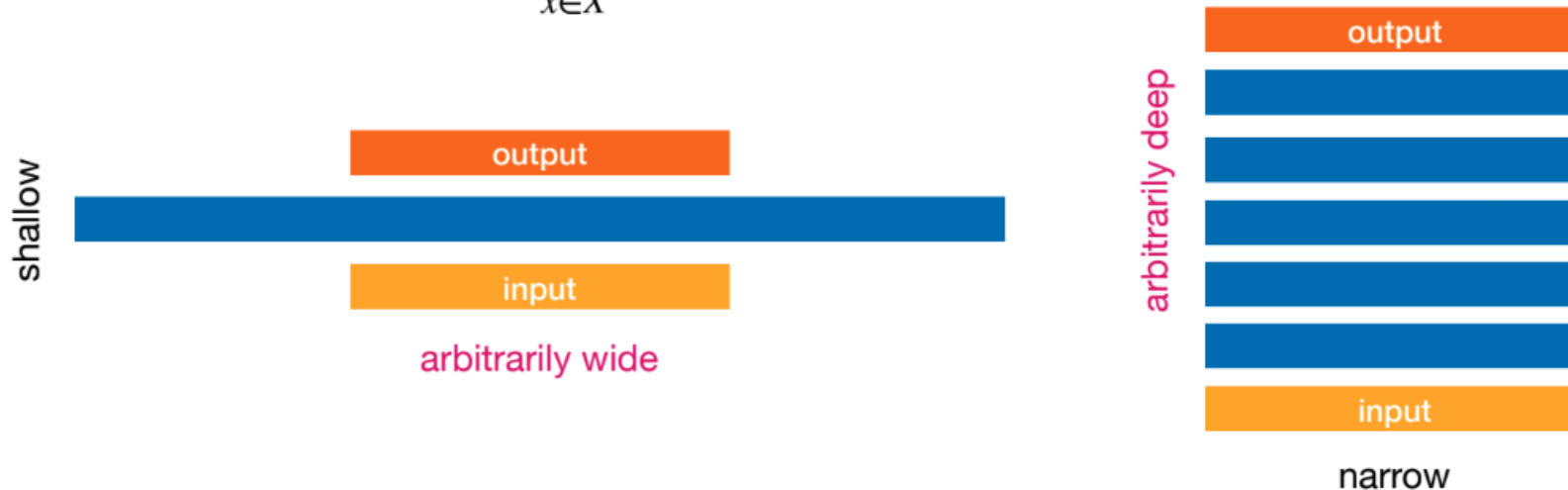


$$y = \sigma(w_1 \cdot \sigma(w_0 \cdot x))$$

DNNs Are Universal Function Approximators

Universal approximation theorem (informal). Given a function $y = f(x)$ and an $\epsilon > 0$, there exists a deep network $y = f_w(x)$ (of arbitrary width or depth) such that:

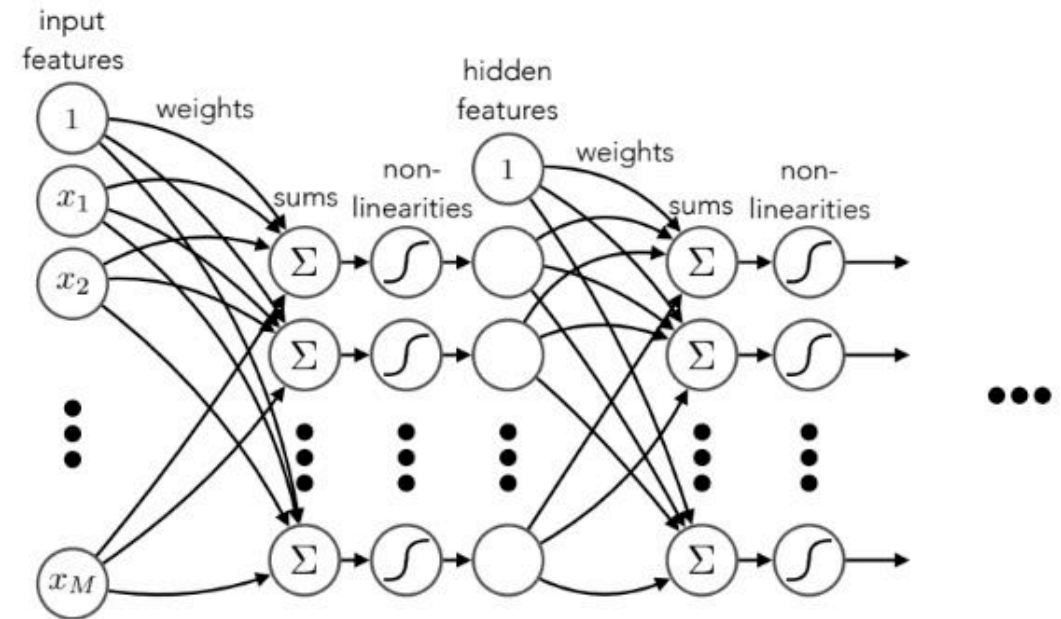
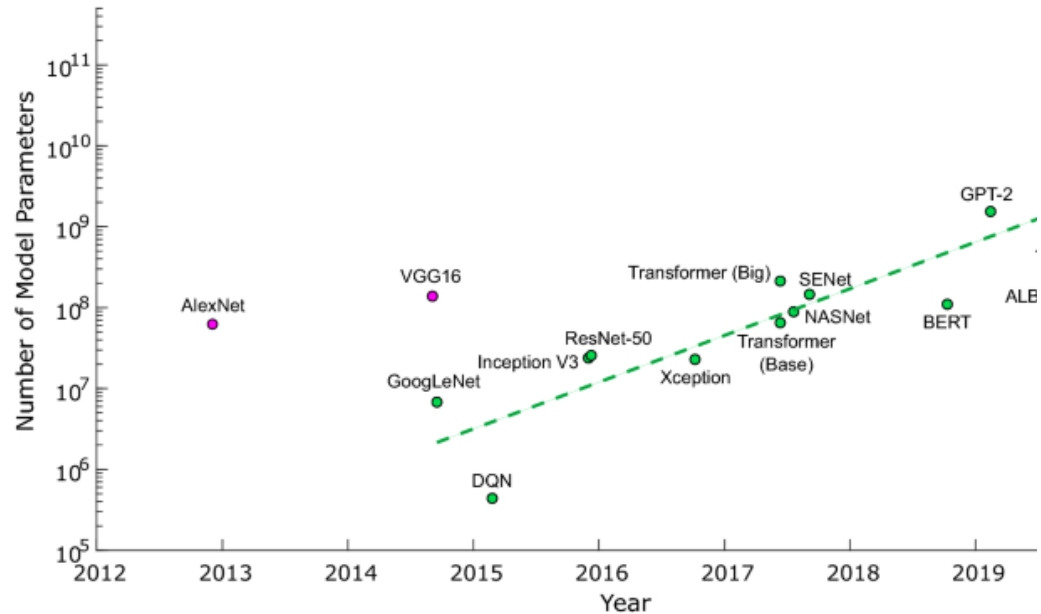
$$\sup_{x \in X} \|f(x) - f_w(x)\| < \epsilon$$



- ❑ Note: Some versions require the true function to be continuous.
- ❑ Note: This means that a network can represent any function, not that it can learn it!
 - The “amount” of function a given network can represent is often called its **expressive power** (formally defined in learning theory).

Training a DNN

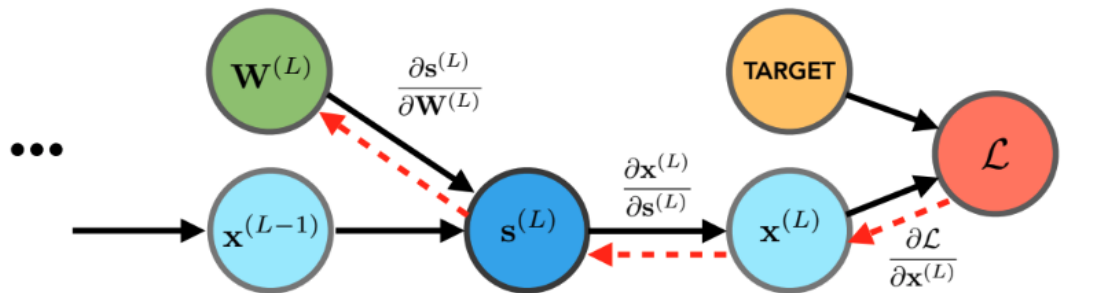
- ❑ Can we still do SGD? $w \leftarrow w - \eta \nabla_w \sum_{i \in \text{Batch}} L(y_i, f(x_i|w))$
- ❑ Yes! Mini batches still work. But how to compute ∇_w ?
- ❑ Problem: w has millions or billions of parameters! We can't naively compute all gradients independently
- ❑ Solution: **backpropagation**



Backpropagation

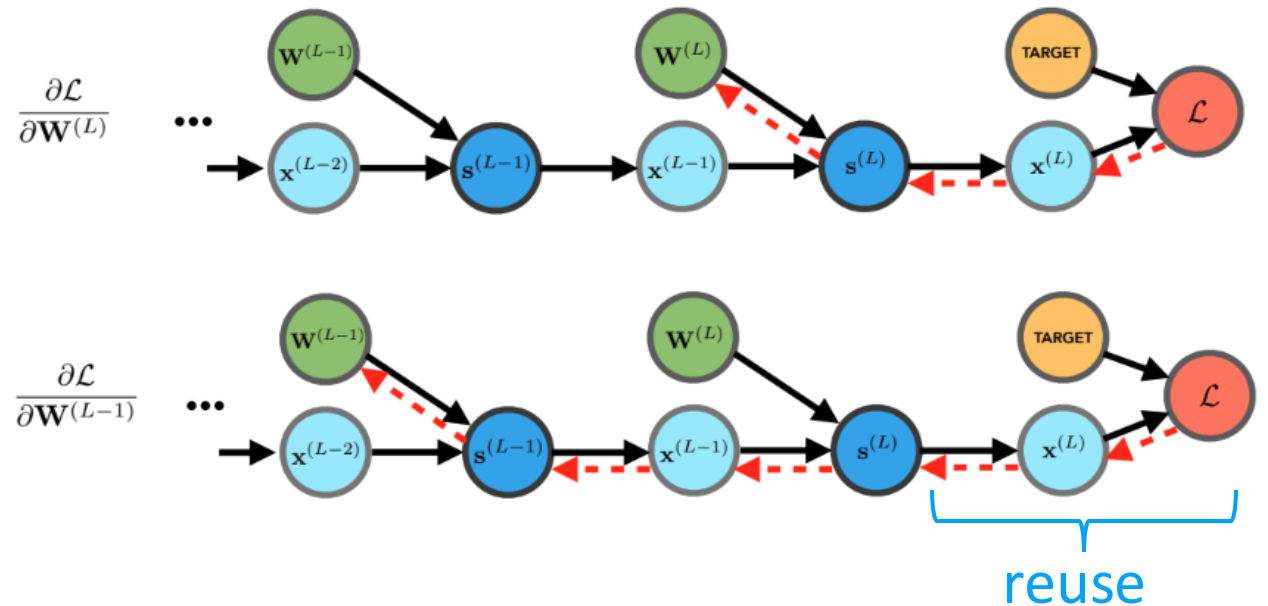
- ❑ DNN is essentially a composition of functions
- ❑ So we can do chain rule!

$$\begin{array}{lll}
 \text{1st layer} & \text{2nd layer} & \text{Loss} \\
 \mathbf{s}^{(1)} = \mathbf{W}^{(1)} \boldsymbol{\tau}_{\mathbf{x}^{(0)}} & \mathbf{s}^{(2)} = \mathbf{W}^{(2)} \boldsymbol{\tau}_{\mathbf{x}^{(1)}} & \dots \\
 \mathbf{x}^{(1)} = \sigma(\mathbf{s}^{(1)}) & \mathbf{x}^{(2)} = \sigma(\mathbf{s}^{(2)}) & \mathcal{L}
 \end{array}$$



$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{W}^{(L)}}$$

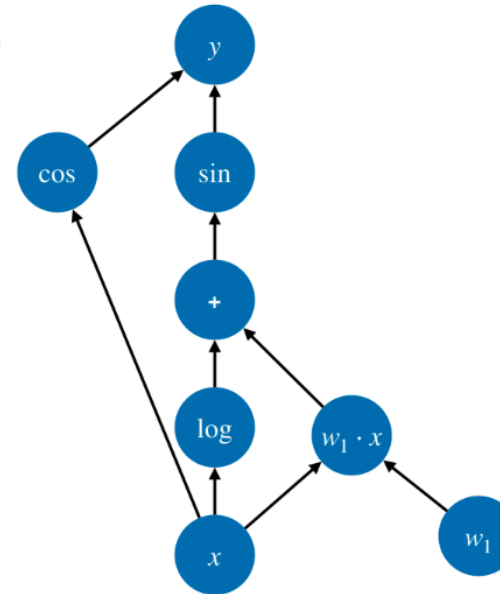
depends on the form of the loss $\rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}}$
 derivative of the non-linearity $\rightarrow \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}$
 $\frac{\partial}{\partial \mathbf{W}^{(L)}} (\mathbf{W}^{(L)} \boldsymbol{\tau}_{\mathbf{x}^{(L-1)}}) = \mathbf{x}^{(L-1)} \boldsymbol{\tau}$



Backpropagation

- ❑ We can use backpropagation to compute the gradients on any **computational graph**

$$y = \sin(w_1 \cdot x + \log(w_2 \cdot x)) + \cos(x)$$



- ❑ Modern deep networks can have a very complex structure!
- ❑ Automatic differentiation in practice (using deep learning software libraries):



PYTORCH

mxnet



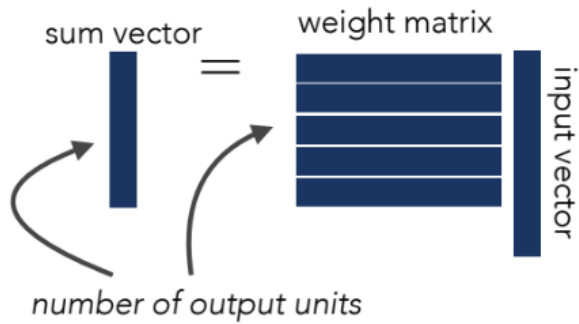
Keras



Caffe2

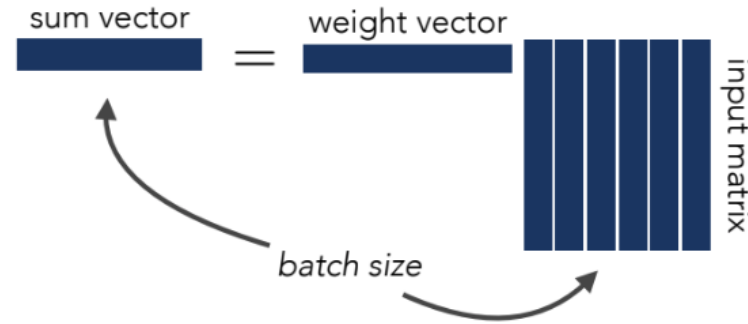
and many more

DNNs are Highly Parallelizable



unit parallelization

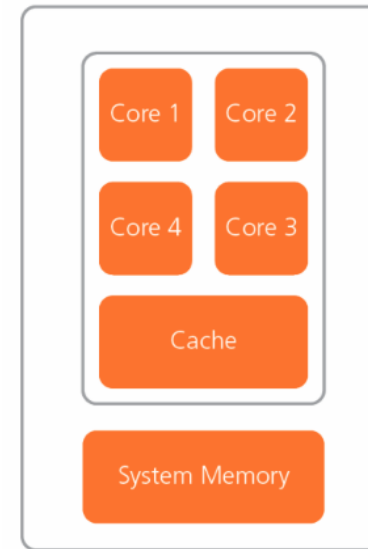
perform all operations within
a layer simultaneously



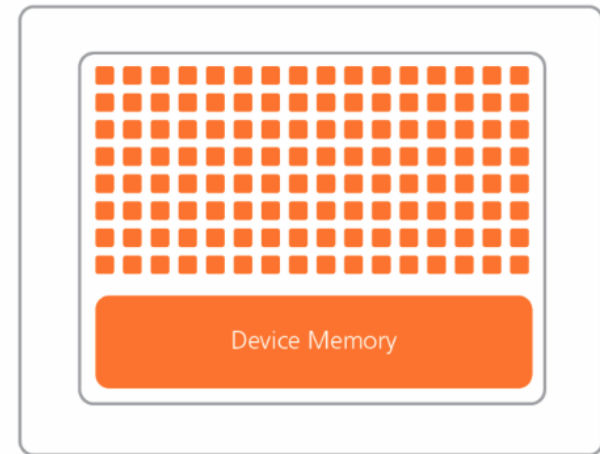
data parallelization

process multiple data examples
simultaneously

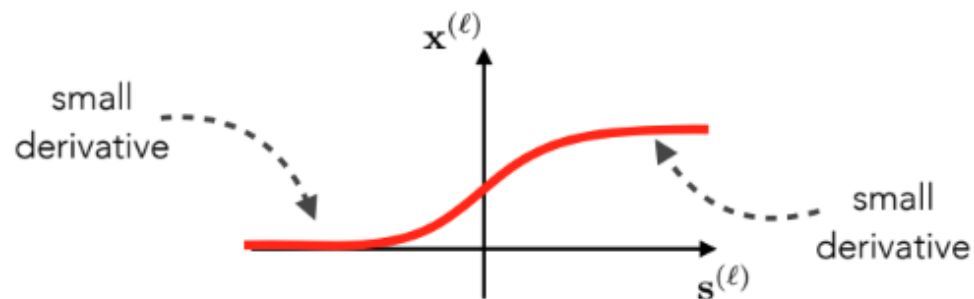
CPU (Multiple Cores)



GPU (Hundreds of Cores)

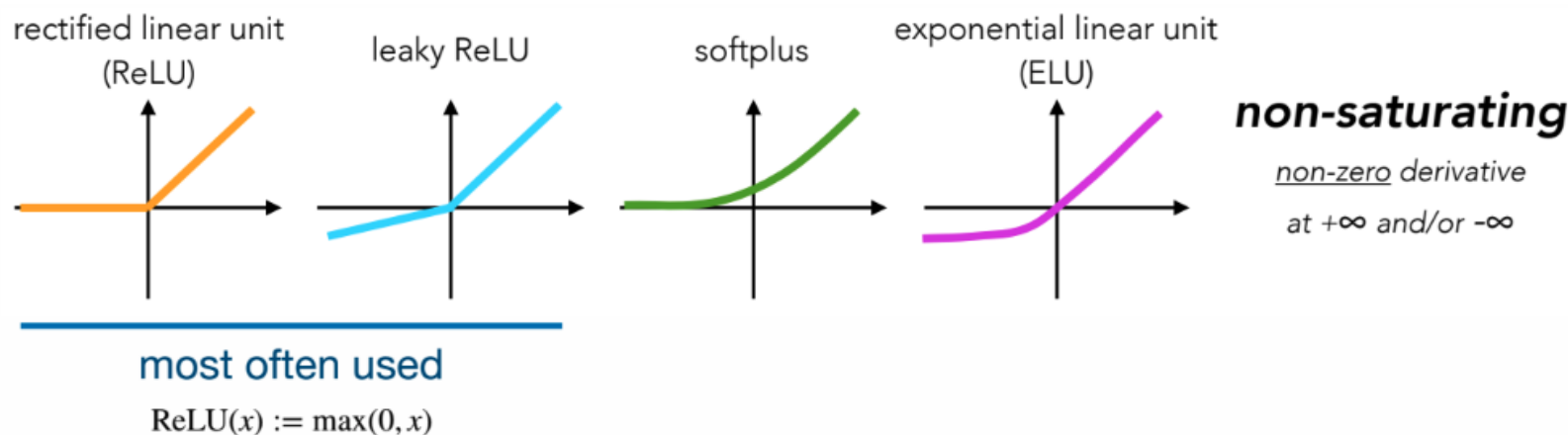


Vanishing Gradients



- ❑ Saturating non-linearities have small derivatives almost everywhere
- ❑ In backprop, the product of many small terms goes to zero!

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \cdots \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}} \cdots \frac{\partial \mathbf{x}^{(L-1)}}{\partial \mathbf{s}^{(L-1)}} \cdots \frac{\partial \mathbf{x}^{(\ell+1)}}{\partial \mathbf{s}^{(\ell+1)}} \cdots \frac{\partial \mathbf{x}^{(\ell)}}{\partial \mathbf{s}^{(\ell)}} \frac{\partial \mathbf{s}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}}$$



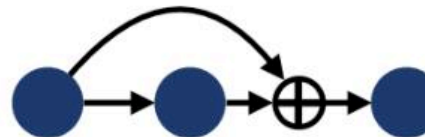
How to Go Deeper

- ❑ Besides non-saturating activations, many techniques have been developed to stabilize deeper neural network training (mitigate vanishing/exploding)
 - Weight initialization
 - Batch normalization
 - Learning rate annealing
 - Residual connections
 - ...
- ❑ Why residual connections?



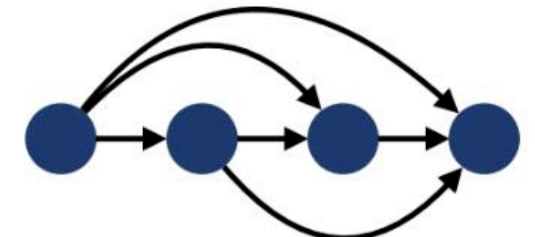
information may not be able to propagate easily
→ *make shorter paths to output*

residual & highway connections



Deep residual learning for image recognition, He et al., 2016
Highway networks, Srivastava et al., 2015

dense (concatenated) connections



Densely connected convolutional networks, Huang et al., 2017

Thank You!